Capítulo 1 — Herramientas matemáticas

Del punto a la matriz de transformación homogénea

Mg. Ing. Juan Pablo Pedroni

BORRADOR

Intencionalmente en blanco

BORRADOR

Índice

1.1 Por qué un robot necesita álgebra 5
El problema: dónde está la herramienta y hacia dónde apunta 5
Dos espacios para describir al robot 5
Posición y orientación: dos preguntas distintas 6
Cuerpos rígidos: lo que se mueve y lo que no se deforma 7
Qué vamos a construir en este capítulo (y qué no) 7
Convenciones de notación 8
1.2 Sistemas de coordenadas 11
El espacio euclídeo tridimensional 11
Versores y la base canónica 11
Sistemas dextrógiros: la regla de la mano derecha 12
Un sistema fijo y muchos móviles 13
1.3 Puntos y vectores 15
Punto, vector posición y vector libre 15
Coordenadas de un punto en un sistema 15
Norma y distancia 16
Producto escalar 17
Producto vectorial 19
1.4 Traslaciones 21
Dos sistemas: uno fijo y otro solidario al objeto 21
Trasladar el objeto 21
Trasladar una figura 21
La otra lectura: mover el punto 23
1.5 Rotaciones 27
Lo que la suma no resuelve: la necesidad de un operador único 27
Dos sistemas con el mismo origen 27
Proyectar sobre los ejes fijos 28
Las columnas de R son los ejes rotados 28
Ejemplo en Octave: rotar un punto y un cuadrado en el plano 29
Las tres matrices de rotación básicas 30
Propiedades de las matrices de rotación 31
Ejemplos en Octave: rotaciones en 3D 32
Rotaciones sucesivas: el producto de matrices 34
El orden importa 34
Respecto del sistema fijo o respecto del sistema móvil 36
1.6 Matriz de transformación homogénea 39
Coordenadas homogéneas 39
Matriz de transformación homogénea 39
La matriz de transformación homogénea 40
¿Qué representa una matriz de transformación homogénea? 42
Los ejes en la mano: sistemas de ejes coordenados físicos 42
Construcción de la matriz: el orden importa 43

BORRADOR

Sucesión de transformaciones: un brazo de dos eslabones....45
Aplicación a figuras: un cuadrado y un círculo....47
Criterios de validación....49
1.7 Ejercicios....51
Vectores (sección 1.3)....51
Traslaciones (sección 1.4)....51
Rotaciones básicas (sección 1.5)....51
Composición de rotaciones: ejes fijos y móviles (sección 1.5)....51
Matriz de transformación homogénea (sección 1.6)....51
El orden de rotación y traslación (sección 1.6)....52
Sucesión de transformaciones (sección 1.6)....52
Figuras (secciones 1.5 y 1.6)....52
Con los sistemas de ejes coordenados físicos (sección 1.6)....52
Respuestas....52

BORRADOR

1.1 Por qué un robot necesita álgebra

El problema: dónde está la herramienta y hacia dónde apunta

Imaginemos que le ordenamos a un robot que abra una puerta cerrada con llave. La tarea parece una sola, pero esconde dos. Primero hay que llevar la llave hasta el ojo de la cerradura: eso es un problema de posición, un punto en el espacio al que hay que llegar. Después hay que introducir la llave en la cerradura, y para eso la llave tiene que estar alineada con la ranura: eso es un problema de orientación. Si el robot llega al punto correcto con la llave atravesada, no abre nada. Si la orienta perfecto pero tres centímetros a la izquierda, tampoco.

Lo mismo pasa con cualquier tarea que le demos a un manipulador. Soldar un cordón exige que la punta del electrodo recorra una curva, pero también que la antorcha se mantenga inclinada a un cierto ángulo respecto de la chapa. Tomar una pieza de una cinta exige llegar hasta ella y, además, presentar la pinza de modo que los dedos queden a los costados y no encima. Pintar, atornillar, insertar un componente en una placa: en todos los casos la pregunta es la misma y tiene dos partes. Dónde está la herramienta y hacia dónde apunta.

Un robot es, mecánicamente, una cadena de cuerpos rígidos (los eslabones) unidos por articulaciones que les permiten girar o deslizarse unos respecto de otros. En el extremo libre de la cadena va la herramienta, y el punto de ella que nos interesa controlar se llama centro de herramienta, o TCP por sus siglas en inglés (Tool Center Point). Cada articulación tiene un motor y un sensor, así que el robot conoce en todo momento el ángulo o el desplazamiento de cada una de sus articulaciones. Decimos que conoce su estado en el espacio de las articulaciones. Lo que el robot no conoce en forma directa es dónde quedó el TCP ni hacia dónde apunta la herramienta como consecuencia de esos ángulos. Es decir, no conoce su estado en el espacio de tareas. Y al revés: el operario sabe a qué punto quiere llegar y con qué orientación, pero no sabe qué ángulos tiene que adoptar cada articulación para lograrlo.

Entre lo que el robot mide (sus articulaciones) y lo que la tarea requiere (una posición y una orientación —es decir, una postura— determinadas para la herramienta) hay que tender un puente. Ese puente es matemático, y construirlo es el objetivo de este capítulo y del siguiente.

[Figura 1.1: Un manipulador de seis articulaciones frente a una cerradura. Se señalan las variables articulares q1 a q6, el TCP en la punta de la llave y el sistema de coordenadas fijo en la base. Dos globos: “esto es lo que el robot mide” sobre las articulaciones y “esto es lo que la tarea pide” sobre la llave.]

Dos espacios para describir al robot

Las dos caras del problema que acabamos de plantear tienen nombre, y conviene fijarlo ahora porque lo vamos a usar en todo el libro.

El espacio de las articulaciones (o espacio articular) es el conjunto de todos los valores que pueden tomar las articulaciones del robot. Si el robot tiene n articulaciones, un punto de ese espacio es un vector q de n componentes: un ángulo por cada articulación de rotación y un desplazamiento por cada articulación prismática. Para un brazo de seis articulaciones rotacionales, q = (q1, q2, q3, q4, q5, q6) son seis ángulos. Decir que el robot conoce su estado en el espacio de las articulaciones es decir que conoce el valor actual de q, que es exactamente lo que le entregan sus sensores.

BORRADOR

El espacio de tareas (o espacio cartesiano, o espacio operacional) es el conjunto de posturas que puede adoptar la herramienta: su posición (x, y, z) y su orientación, respecto de un sistema de referencia fijo, en general el de la base. Un punto de ese espacio es una postura del TCP. Es el espacio en el que se expresan las tareas, porque la pieza, la cerradura o el cordón de soldadura están en el mundo, no en los ángulos del robot. El estado en el espacio de tareas es la postura actual del TCP, y es lo que el robot no mide en forma directa.

La relación entre los dos espacios es el tema de los capítulos 2, 4 y 5: pasar de q a la postura del TCP es la cinemática directa, y pasar de la postura del TCP a q es la cinemática inversa.

Queda una tercera noción, que viene de la teoría de control y que conviene distinguir de las dos anteriores. Cuando hablamos del estado del robot en el sentido de variables de estado, no alcanza con conocer la posición de las articulaciones: hace falta también conocer su velocidad. El estado del robot es entonces el par (q, q̇), posiciones y velocidades articulares, 2n números en total. Son las variables que hay que conocer en un instante para poder predecir cómo va a seguir moviéndose el robot si conocemos las fuerzas y los torques que actúan sobre él, y son las que usa cualquier controlador de movimiento para decidir qué hacer con los motores.

En este libro vamos a trabajar casi siempre con la posición sola, q, porque la cinemática es geometría: describe dónde están las cosas, no cómo llegaron ni con qué fuerza. Las velocidades articulares aparecen recién en el capítulo 3, con el Jacobiano, que es justamente la herramienta que relaciona q̇ con la velocidad del TCP. Cuando en los capítulos de trayectorias generemos perfiles de posición y velocidad para cada articulación, lo que estaremos generando es, precisamente, la secuencia de estados (q, q̇) que el controlador del robot debe seguir.

Posición y orientación: dos preguntas distintas

Vale la pena detenerse en la diferencia, porque buena parte de la dificultad de la robótica viene de tratar las dos preguntas con la misma herramienta.

La posición es la más intuitiva. Para decir dónde está un punto en el espacio alcanzan tres números: sus coordenadas x, y, z respecto de algún sistema de referencia. Si el sistema está en la base del robot, decir que el TCP está en (450, 200, 850) milímetros es decir todo lo que hay que decir sobre su posición.

[Figura 1.2: Modelo en 3D de un robot, sobre un fondo de un sistema de ejes en 3D, indicando las coordenadas x y z del TCP.]

La orientación es más esquiva. Una herramienta no es un punto: tiene una dirección de aproximación (el eje de la llave, el eje del taladro, la dirección en que avanza la pinza para cerrarse sobre la pieza) y tiene un “arriba” (la ranura de la llave, el plano de los dedos de la pinza). Para describir cómo está orientada hay que decir hacia dónde apunta cada uno de esos ejes, y eso también lleva tres números, aunque de una naturaleza distinta a los de la posición: son ángulos, no distancias, y se combinan de una manera que no es tan inocente como sumar. Girar 90 grados alrededor de un eje y luego 90 alrededor de otro no da lo mismo que hacerlo en el orden inverso. Podemos comprobarlo con un libro: Si lo acostamos sobre la mesa con la tapa hacia arriba, lo giramos un cuarto de vuelta alrededor del eje vertical y después lo paramos sobre el lomo. Ahora, volvamos a la posición inicial y hagamos las dos operaciones al revés. El libro termina en dos lugares distintos.

BORRADOR

[Figura 1.3: El experimento del libro. Dos filas de tres dibujos: arriba, rotación alrededor del eje vertical seguida de rotación alrededor del eje horizontal; abajo, las mismas rotaciones en orden inverso. El resultado final es distinto.]

A lo largo de este capítulo aprenderemos que las traslaciones se suman y conmutan, mientras que las rotaciones se multiplican y no conmutan. Un robot que pueda ubicar su herramienta en cualquier posición y con cualquier orientación necesita en teoría seis grados de libertad: tres para la posición y tres para la orientación. Por eso los robots industriales de uso general tienen seis articulaciones, ni cinco ni siete. Con menos, hay orientaciones que no se alcanzan; con más, sobran soluciones y hay que elegir.

En este capítulo vamos a presentar una única representación matemática que contiene a las dos cosas, posición y orientación, y que además se puede encadenar: que la representación de la herramienta respecto del último eslabón, compuesta con la del último eslabón respecto del anterior, y así hasta la base, dé la representación de la herramienta respecto de la base. Esa representación existe, se llama matriz de transformación homogénea, y es a donde apunta todo el capítulo.

Cuerpos rígidos: lo que se mueve y lo que no se deforma

Hay una hipótesis que vamos a usar sin repetirla en cada renglón: todo lo que se mueve en este libro es un cuerpo rígido: un conjunto de puntos cuyas distancias mutuas no cambian. Los eslabones del robot son rígidos, la herramienta es rígida, la pieza que agarra es rígida.

La consecuencia práctica es enorme. Si un cuerpo es rígido, para saber dónde quedó cada uno de sus infinitos puntos después de moverlo alcanza con saber dónde quedó un punto de referencia (su posición) y cómo giró el cuerpo alrededor de ese punto (su orientación). No hay que seguir a cada punto por separado: todos se mueven juntos, solidarios. Eso es lo que permite asociar a cada eslabón un sistema de ejes coordenados que viaja con él, y describir el movimiento del eslabón como el movimiento de ese sistema. Si el sistema está acá y está orientado así, cada punto del eslabón está donde tiene que estar.

Los movimientos que preservan las distancias entre puntos se llaman movimientos rígidos, y son exactamente dos, y sus combinaciones: trasladar y rotar. No hay otros. Estirar, comprimir, torcer o reflejar no son movimientos rígidos, y por lo tanto no aparecen en robótica. Esto simplifica el álgebra que viene: de todas las transformaciones lineales posibles, solo nos van a interesar las que trasladan y las que rotan.

La hipótesis no es gratuita. Un brazo real se flexiona bajo carga, los engranajes tienen juego, la pinza aprieta y deforma la pieza. Pero para los robots industriales y para los problemas de cinemática que trata este libro, esas deformaciones son chicas comparadas con los movimientos que queremos describir, y se las deja para el modelo dinámico o para la calibración. En otras áreas, como la animación por computadora, la deformación es justamente lo que se busca para dar realismo; acá la descartamos a propósito.

Qué vamos a construir en este capítulo (y qué no)

El capítulo arranca de cero. No requiere conocimiento más allá del álgebra de vectores y matrices al nivel de un curso inicial de ingeniería: con saber qué es un producto escalar, qué es un producto vectorial y cómo se multiplican dos matrices y una matriz y un vector es suficiente. Todo lo demás se construye acá.

BORRADOR

El recorrido es el siguiente. Primero vamos a fijar con precisión qué entendemos por sistema de coordenadas, porque sobre esa definición se apoya todo lo demás y las convenciones que elijamos (en particular el sentido de giro positivo) nos van a perseguir hasta el último capítulo. Después vamos a describir las traslaciones y las rotaciones, primero en el plano y después en el espacio. Vamos a ver cómo se componen las rotaciones y por qué el orden importa. Y entonces vamos a dar el paso que justifica el capítulo: introducir las coordenadas homogéneas, un truco que agrega una cuarta componente a los vectores de tres dimensiones y que permite escribir una traslación y una rotación como una única multiplicación de matrices. El resultado es la matriz de transformación homogénea, una matriz cuadrada de cuatro por cuatro que describe al mismo tiempo dónde está y cómo está orientado un sistema de coordenadas respecto de otro, y que sigue las reglas ordinarias del álgebra de matrices.

Lo que este capítulo no hace también conviene dejarlo claro. No describe robots: no hay articulaciones, ni eslabones, ni parámetros geométricos de ningún brazo en particular. Eso es el capítulo 2, donde la matriz homogénea que construimos acá se convierte en la herramienta para modelar cadenas cinemáticas con el algoritmo de Denavit y Hartenberg. Tampoco trata otras formas de representar una orientación, como los ángulos de Euler, los ángulos roll-pitch-yaw o los cuaterniones: el capítulo se concentra en la matriz de rotación, que es la representación que la matriz homogénea lleva adentro y la que vamos a usar en todo el resto del libro. Y no hay velocidades: todo lo que sigue es geometría pura, posiciones y orientaciones congeladas en el tiempo. El movimiento, en el sentido de cómo cambian esas cantidades, recién aparece con el Jacobiano en el capítulo 3.

Y una aclaración de alcance que vale para toda la primera parte del libro: los robots de los capítulos 1 a 7 son manipuladores de base fija. Todo lo que digamos sobre “la base” y sobre el sistema de referencia que lleva asociado supone que esa base no se mueve. Los robots móviles, donde la base es justamente lo que se desplaza, tienen su propio tratamiento en los capítulos 8 y 9, y allí veremos qué cambia y qué se conserva de la matemática que construimos acá.

Convenciones de notación

Para no tener que aclararlo en cada página, fijamos acá las convenciones que usa todo el libro, texto y código.

- Los sistemas de coordenadas se escriben entre llaves: {S0}, {Si}, {S_TCP}.
- Los ejes de un sistema son x, y, z; cuando hace falta distinguir de qué sistema son, llevan el subíndice del sistema: x1, y1, z1 son los ejes de {S1}. Cuando en un mismo problema hay un sistema fijo y otro solidario a un objeto, para no cargar todo de subíndices llamamos x, y, z a los ejes del fijo y u, v, w a los del objeto. En las figuras, el eje x se dibuja en rojo, el y en verde y el z en azul, que es la convención de la mayoría de los simuladores y la que usa el código que acompaña al libro.
- Los vectores y puntos van en negrita cursiva minúscula en el texto (p, v) y son vectores columna en las fórmulas y en el código. Sus componentes van en cursiva con subíndice: px, py, pz.
- Las matrices van en mayúscula (R, T). Cuando una matriz relaciona dos sistemas, lleva los dos como índices: la matriz que expresa {Sj} respecto de {Si} se escribe con i arriba a la izquierda y j abajo a la derecha, y se lee “j respecto de i”. La notación exacta se introduce con la matriz

BORRADOR

de transformación homogénea, en la sección 1.6. Las rotaciones básicas se escriben indicando el eje de giro y el ángulo: R(x, α), R(y, φ), R(z, θ).

- Los ángulos se miden en grados sexagesimales y las longitudes en milímetros. Es lo que marcan los catálogos de los robots industriales y lo que espera todo el código del libro (las funciones usan cosd y sind). Donde haga falta pasar a radianes para una cuenta, se dice explícitamente.
- Los ángulos positivos son los que respetan la regla de la mano derecha alrededor del eje de giro, sin excepción.

Elegir una convención y no soltarla es, en robótica, media batalla ganada.

BORRADOR

Intencionalmente en blanco

BORRADOR

1.2 Sistemas de coordenadas

El espacio euclídeo tridimensional

Un robot trabaja en un volumen: el espacio que alcanza su herramienta tiene largo, ancho y alto. Para determinar dónde se encuentra un punto de ese volumen necesitamos tres números, y para que esos tres números signifiquen algo necesitamos antes un sistema de referencia: un origen y tres direcciones a lo largo de las cuales medir. De todas las maneras posibles de organizar un espacio de tres dimensiones, la que usamos en robótica es la más simple y la más antigua: el espacio euclídeo con coordenadas cartesianas, donde la posición de un punto queda determinada por tres distancias medidas sobre tres ejes.

Existen otros sistemas de coordenadas (polares, cilíndricas, esféricas) y hay robots cuya geometría invita a usarlos: un brazo cilíndrico se describe cómodamente con un ángulo, un radio y una altura. Pero el precio de esa comodidad es que las fórmulas cambian de un robot a otro. Las coordenadas cartesianas tienen la virtud de que sirven para todos, y de que las operaciones que vamos a necesitar (trasladar, rotar, componer, concatenar) se escriben con álgebra de matrices ordinaria. Por eso, de acá en adelante, cuando hablemos de un sistema de coordenadas sin más aclaración, vamos a estar hablando de uno cartesiano con las siguientes propiedades:

- Rectilíneo: sus tres ejes son rectas.
- Ortogonal: cada eje es perpendicular a los otros dos.
- Normalizado: la unidad de medida es la misma sobre los tres ejes, y el vector que marca la dirección de cada eje tiene longitud uno.
- Dextrógiro: el tercer eje queda definido por los dos primeros según la regla de la mano derecha, que vemos enseguida.

Las tres primeras propiedades son las que uno espera de cualquier terna de ejes bien dibujada. La cuarta es la que trae problemas si se la descuida, y por eso le dedicamos una sección aparte.

[Figura 1.4: Un sistema de coordenadas cartesiano con su origen O, los ejes x, y, z, y un punto p con sus tres coordenadas marcadas como proyecciones sobre los ejes.]

Versores y la base canónica

Un versor es un vector de longitud uno. Cada eje de nuestro sistema tiene un versor asociado, que apunta en el sentido positivo del eje y que marca, de paso, la unidad de medida. A los tres versores del sistema los vamos a llamar ***i***, ***j*** y ***k***, para los ejes *x*, *y* y *z* respectivamente, y en coordenadas se escriben así:

$$i=\begin{bmatrix}1\\0\\0\end{bmatrix}, j=\begin{bmatrix}0\\1\\0\end{bmatrix}, k=\begin{bmatrix}0\\0\\1\end{bmatrix}$$

El conjunto {***i***, ***j***, ***k***} se llama base canónica, y su gracia es que cualquier punto ***p*** del espacio se puede escribir como una combinación lineal de los tres: tanto de ***i***, más tanto de ***j***, más tanto de ***k***. Los tres "tantos" son las coordenadas del punto.

BORRADOR

Definición: base y base canónica

Una base de un espacio vectorial es un conjunto de vectores con dos propiedades: son linealmente independientes (ninguno se puede escribir como combinación de los otros) y generan el espacio (cualquier vector se puede escribir como combinación lineal de ellos). En el espacio tridimensional toda base tiene exactamente tres vectores, y los coeficientes de la combinación que reconstruye un vector dado son sus coordenadas en esa base. Cambiar de base es cambiar de coordenadas sin mover el vector.

La base canónica es la base particular formada por los versores de los tres ejes del sistema de coordenadas, {***i***, ***j***, ***k***}. Es ortonormal (sus vectores son perpendiculares entre sí y de longitud uno) y dextrógira, y tiene la propiedad de que las coordenadas de un vector en ella coinciden con sus componentes cartesianas: no hay que resolver ningún sistema de ecuaciones para encontrarlas. Toda otra terna ortonormal dextrógira que aparezca en el libro (los ejes de un sistema girado, por ejemplo) es también una base, y expresar un vector en ella es exactamente el problema que resuelve una matriz de rotación.

$$p = p_x\, i + p_y\, j + p_z\, k = \begin{pmatrix} p_x \\ p_y \\ p_z \end{pmatrix}$$

Esta manera de ver un punto, como suma de tres pasos a lo largo de tres direcciones, parece una formalidad pero es la clave de todo lo que sigue. Cuando más adelante un sistema de coordenadas gire respecto de otro, sus versores van a dejar de coincidir con ***i***, ***j***, ***k***, y la pregunta “cuáles son las coordenadas del punto en el sistema girado” se va a responder exactamente con esta descomposición: proyectando el punto sobre los versores nuevos.

En Octave un punto es simplemente un vector columna de tres componentes, y la base canónica es la matriz identidad:

```
p = [450; 200; 850];      % un punto, en mm
B = eye(3);               % columnas: i, j, k
i = B(:,1); j = B(:,2); k = B(:,3);
p_check = p(1)*i + p(2)*j + p(3)*k;   % reconstruir p desde la base
disp(p_check - p)         % valor esperado: [0; 0; 0]
```

Sistemas dextrógiros: la regla de la mano derecha

Dados dos ejes perpendiculares, *x* e *y*, hay dos maneras de definir la orientación de un tercero perpendicular a ambos: “hacia un lado” o “hacia el otro”. Las dos ternas resultantes son imágenes especulares una de la otra, y no hay rotación que lleve una sobre la otra, así que hay que elegir. La convención universal en robótica, en física y en casi toda la ingeniería es la terna dextrógira, o “de mano derecha”.

La regla práctica para construirla es la del tirabuzón. Se fija el eje *x*. Se fija el eje *y*, perpendicular a *x*. Después se imagina que se pliega el eje *x* sobre el eje *y*, girando por el camino más corto (un cuarto de vuelta), y se hace girar un tirabuzón en ese mismo sentido: la dirección en la que avanza el tirabuzón es el sentido positivo de *z*. Con la mano derecha es lo mismo: los dedos se curvan de *x* hacia *y*, y el pulgar extendido apunta según *z*.

[Figura 1.5: La regla de la mano derecha. A la izquierda, la mano con los dedos curvados de x hacia y y el pulgar apuntando según z. A la derecha, el tirabuzón girando de x hacia y y avanzando según z.]

La misma regla, escrita en álgebra, es el producto vectorial: en una terna dextrógira se cumple que ***k*** es el producto vectorial de ***i*** por ***j***, y las otras dos relaciones salen permutando cíclicamente los ejes.

$$i \times j = k\,,\; j \times k = i\,,\; k \times i = j$$

El orden de los factores importa: ***j*** por ***i*** es igual a menos ***k***. Esa es justamente la terna especular, la levógira, que no vamos a usar nunca. Y la regla vale también para el sentido positivo de giro alrededor de cada eje: una rotación positiva alrededor de *z* es la que lleva *x* hacia *y*, una rotación positiva alrededor de *x* lleva *y* hacia *z*, y una rotación positiva alrededor de *y* lleva *z* hacia *x*. Cuando en la sección 1.5 escribamos las matrices de rotación básicas, los signos de los senos van a salir de acá y de ningún otro lado.

Que una terna sea dextrógira no dice nada sobre cómo está orientada en el espacio. El eje *z* puede apuntar hacia arriba, hacia el costado o hacia el lector; lo único que fija la regla es la relación entre los tres ejes. En un robot vamos a tener un sistema en la base, uno en cada eslabón y uno en la herramienta, todos dextrógiros y todos orientados de manera distinta, y una fuente importante de los errores de signo que cometen quienes empiezan con la cinemática viene de dibujar una terna, darla por dextrógira sin comprobarlo, y arrastrar un eje al revés durante páginas.

[Figura 1.6: Cuatro ternas dextrógiras orientadas de maneras distintas: z hacia arriba, z hacia el lector, z hacia abajo, y una terna inclinada. En todas se cumple la regla de la mano derecha.]

El *sanity check* es inmediato en Octave. Dados tres versores candidatos, el producto vectorial de los dos primeros tiene que coincidir con el tercero; y, equivalentemente, el determinante de la matriz que los tiene como columnas tiene que valer +1 (una terna levógira tiene por resultado −1, y una que no sea ortonormal da cualquier otro valor).

```
x = [1; 0; 0];
y = [0; 1; 0];
z = cross(x, y)          % valor esperado: [0; 0; 1]
B = [x y z];
det(B)                   % valor esperado: +1

% Terna levógira: z apuntando al revés
B_mala = [x y -z];
det(B_mala)              % valor esperado: -1 (no es dextrógira)
```

Criterio de validación: una matriz 3×3 cuyas columnas son los versores de una terna representa un sistema dextrógiro válido si y solo si es ortogonal (cada columna tiene norma uno y las columnas son perpendiculares entre sí) y su determinante vale +1. Guardemos esta observación: es exactamente la condición que vamos a exigirle a una matriz de rotación.

Un sistema fijo y muchos móviles

En un robot no hay un solo sistema de coordenadas sino varios, y conviene presentarlos ahora aunque recién los usemos en el capítulo 2.

Una aclaración de alcance antes de seguir: lo que sigue vale para robots manipuladores de base fija, que son el objeto de la primera parte del libro. En un robot móvil la base también se mueve, así que el sistema de referencia “fijo” ya no puede estar en el robot: hay que ponerlo en el mundo (el piso, el taller) y la posición y orientación del vehículo respecto de él pasan a ser parte del problema. Ese caso se trata en los capítulos 8 y 9; hasta entonces, cuando digamos “la base”, es una base que no se mueve.

El sistema de ejes coordenados de la base, que llamaremos $\{S_0\}$, está fijo al suelo, a la mesa o a la estructura sobre la que está montado el robot. Es el sistema de referencia por excelencia: la posición y orientación de la herramienta, de las piezas y de los obstáculos se expresan, en última instancia, respecto de él. Cuando digamos "el TCP está en (450, 200, 850)" sin más aclaración, las coordenadas están medidas en $\{S_0\}$.

A cada eslabón i del robot le asociamos un sistema propio, $\{S_i\}$, que viaja solidario con él. Como el eslabón es rígido, conocer dónde está y cómo está orientado $\{S_i\}$ equivale a conocer dónde está cada punto del eslabón. Y como cada articulación mueve un eslabón respecto del anterior, el movimiento del robot se describe como una cadena de sistemas, $\{S_0\}$, $\{S_1\}$, $\{S_2\}$, …, $\{S_{i-1}\}$, $\{S_i\}$, …, $\{S_n\}$, cada uno moviéndose respecto del que lo precede. En la punta de la cadena va el sistema de la herramienta, cuyo origen es el TCP.

La pregunta central de la cinemática, dicha en estos términos, es: conocidas las posiciones relativas entre cada par de sistemas consecutivos, cómo se expresa un punto dado en el sistema de la herramienta en coordenadas del sistema de la base. Para responderla necesitamos saber trasladar y rotar un sistema respecto de otro, y encadenar esas operaciones. Eso es lo que construye el resto del capítulo.

[Figura 1.7: Un manipulador de tres eslabones con los sistemas $\{S_0\}$ en la base, $\{S_1\}$, $\{S_2\}$ y $\{S_3\}$ en cada eslabón, y el sistema de la herramienta en el TCP. Todos dextrógiros, todos orientados de manera distinta.]

BORRADOR

1.3 Puntos y vectores

Punto, vector posición y vector libre

En el lenguaje cotidiano “punto” y “vector” se usan casi como sinónimos, y al codificar los dos se definen de la misma manera: tres números en una columna. Pero representan cosas distintas, y en robótica la diferencia va a importar en cuanto empecemos a rotar y trasladar.

Un punto es un lugar del espacio. No tiene tamaño ni dirección: es la punta de la llave, el centro de un tornillo, el TCP. Existe independientemente de cualquier sistema de coordenadas; lo que depende del sistema es la manera de describir su posición.

Dicho de otra manera: el punto está donde está, sin importar el sistema de coordenadas que elijamos de referencia; lo que cambia de un sistema a otro son los tres números (sus coordenadas) con que lo describimos. Si movemos el origen de los ejes desde la base del robot hasta una esquina de la mesa, la punta de la llave no se mueve un milímetro, pero sus coordenadas cambian, porque ahora las medimos desde otro lugar.

[Figura 1.8: Dos sistemas de ejes coordenados en distinta posición y orientación, indicando en ambos las coordenadas a un único punto en el espacio]

Un vector es un segmento orientado: tiene módulo (una longitud), dirección (una recta) y sentido (hacia dónde apunta sobre esa recta). Hay dos clases de vectores que conviene distinguir desde el principio.

El vector posición de un punto P es el vector que va desde el origen O del sistema de coordenadas hasta P. Lo escribimos $\boldsymbol{p}$. Está atado al origen: si cambiamos de sistema, el punto sigue en el mismo lugar pero su vector posición cambia, porque cambió el origen desde donde se lo mide. Es la manera de “convertir un punto en números”.

[Figura 1.9: Dos sistemas de ejes coordenados en distinta posición y orientación, graficando dos vectores p ambos apuntando al mismo punto en el espacio]

Un vector libre, en cambio, no está atado a ningún punto: representa un desplazamiento, una dirección o una velocidad, y es el mismo vector lo dibujemos donde lo dibujemos. El vector que va de P a Q, $\boldsymbol{v} = \boldsymbol{q} - \boldsymbol{p}$, es un vector libre: dice “tanto en x, tanto en y, tanto en z”, sin importar dónde arranca. Los versores $\boldsymbol{i}$, $\boldsymbol{j}$, $\boldsymbol{k}$ de la sección anterior son vectores libres, y también lo es el eje de una articulación o la dirección en que apunta una herramienta.

*[Figura 1.10: Un sistema de coordenadas con dos puntos P y Q. Desde el origen salen sus vectores posición **p** y **q**. Entre P y Q, el vector libre **v** = **q** − **p**, dibujado también trasladado a otro lugar para mostrar que es el mismo vector.]*

Coordenadas de un punto en un sistema

Las coordenadas de un punto P en un sistema $\{S\}$ son las componentes de su vector posición sobre la base de ese sistema: los tres números p_x, p_y, pz tales que $\boldsymbol{p} = p_x\,\boldsymbol{i} + p_y\,\boldsymbol{j} + \text{pz}\,\boldsymbol{k}$. Geométricamente son las proyecciones ortogonales del segmento OP sobre cada eje.

La palabra clave es “en un sistema”. Un mismo punto tiene coordenadas distintas en sistemas distintos, y cuando haya más de uno en juego (que en robótica es habitual) hay que indicar respecto de cuál se miden. Para eso agregamos un superíndice a la izquierda con el número del sistema:

BORRADOR

$$ {}^0p = \begin{bmatrix} {}^0p_x \\ {}^0p_y \\ {}^0p_z \end{bmatrix} \text{coordenadas de } P \text{ medidas en } \{S_0\} $$

Cuando el superíndice no aparece, el sistema es el de la base, $\{S_0\}$. Así, ${}^0\boldsymbol{p}$ y ${}^1\boldsymbol{p}$ son dos descripciones del mismo punto, pero desde dos sistemas distintos, y encontrar una a partir de la otra es exactamente el problema que vamos a resolver con la matriz de transformación homogénea.

[Figura 1.11: Un punto P y dos sistemas, $\{S_0\}$ y $\{S_1\}$, con orígenes distintos y orientaciones distintas. Se dibujan los dos vectores posición, ${}^0\boldsymbol{p}$ desde O_0 y ${}^1\boldsymbol{p}$ desde O_1, que apuntan al mismo punto pero tienen componentes diferentes.]

En Octave un punto es un vector columna de tres componentes y una nube de puntos (el contorno de una pieza, una trayectoria, los vértices de una figura) es una matriz de tres filas con un punto por columna. Esta convención, una columna por punto, es la que permite después transformar la figura entera con una sola multiplicación de matrices.

```
P = [100;...
     50;...
     0];              % un punto, en mm
fig = [  0 100 100   0   0;   % un cuadrado de 100 mm en el plano xy
         0   0 100 100   0;   % (el último punto repite el primero
         0   0   0   0   0];  %  para cerrar el dibujo)
disp(P)                       % el punto, como vector columna
disp(fig)                     % la figura: un punto por columna
size(fig)                     % valor esperado: 3   5
plot3(fig(1,:), fig(2,:), fig(3,:), 'b-o'); grid on; axis equal
xlabel('x [mm]'); ylabel('y [mm]'); zlabel('z [mm]')
```

La consola de Octave muestra el punto como una columna de tres números y la figura como una matriz de tres filas, donde cada columna es uno de sus vértices:

```
   100
    50
     0

     0   100   100     0     0
     0     0   100   100     0
     0     0     0     0     0
```

Norma y distancia

La norma (o módulo) de un vector es su longitud. En un sistema cartesiano ortonormal se calcula con el teorema de Pitágoras aplicado dos veces:

$$ \| v \| = \sqrt{v_x^2 + v_y^2 + v_z^2} $$

Por qué "dos veces": primero se proyecta el vector sobre el plano *xy*; esa sombra es la hipotenusa de un triángulo rectángulo de catetos v_x y v_y, así que mide $\sqrt{(v_x^2 + v_y^2)}$. Después, el vector completo es la hipotenusa de otro triángulo rectángulo, vertical, cuyos catetos son esa sombra y la componente *vz*. Aplicando Pitágoras otra vez queda la raíz de la suma de los tres cuadrados.

*[Figura 1.12: Un vector **v** en el espacio, su proyección sobre el plano xy y los dos triángulos rectángulos: el horizontal, de catetos v^x y v_y, y el vertical, de catetos la proyección y vz.]*

La distancia entre dos puntos *P* y *Q* es la norma del vector libre que los une, $\boldsymbol{q} - \boldsymbol{p}$. Un vector de norma uno es un versor, y cualquier vector no nulo se convierte en versor dividiéndolo por su norma;

a esa operación se la llama normalizar, y la vamos a usar para obtener la dirección de un eje o de una herramienta sin que importe su longitud.

$$d(P,Q) = \|q - p\|, \hat{u} = \frac{v}{\|v\|}$$

Hay una propiedad de la norma que es la definición misma de lo que dijimos en la sección 1.1 sobre cuerpos rígidos: los movimientos rígidos son exactamente las transformaciones que conservan las distancias. Una traslación o una rotación cambian las coordenadas de dos puntos, pero no la distancia entre ellos. Esto nos da un ***sanity check*** muy barato para cualquier transformación que escribamos: si después de aplicarla la distancia entre dos puntos de un eslabón cambió, la transformación está mal.

```
p = [100; 50; 0];
q = [400; 450; 0];
v = q - p;
d = norm(v)                  % valor esperado: 500
u = v / norm(v)             % valor esperado: [0.6; 0.8; 0]
norm(u)                      % valor esperado: 1
```

Producto escalar

El producto escalar de dos vectores ***a*** y ***b*** es un número, y se puede definir de dos maneras equivalentes. La primera es geométrica: el producto de las normas por el coseno del ángulo θ que forman. La segunda es algebraica: la suma de los productos de las componentes homólogas.

$$a \cdot b = \|a\| \|b\| \cos\theta = a_x b_x + a_y b_y + a_z b_z = a^T b$$

Un ejemplo numérico muestra que las dos definiciones dan lo mismo. Tomemos dos vectores de 100 mm en el plano *xy*: ***a*** sobre el eje *x* y ***b*** a 60° de él. Calculado componente a componente, con la notación matricial $\boldsymbol{a}^T\boldsymbol{b}$, con la función dot de Octave, o con normas y coseno, el resultado es el mismo; y de vuelta, despejando el coseno se recupera el ángulo.

```
a = [100; 0; 0];                    % 100 mm sobre el eje x
b = [50; 50*sqrt(3); 0];            % 100 mm, a 60 grados de a

a(1)*b(1) + a(2)*b(2) + a(3)*b(3)   % valor esperado: 5000 (componente a
componente)
a' * b                              % valor esperado: 5000 (fila por columna)
dot(a, b)                           % valor esperado: 5000 (función de Octave)
norm(a) * norm(b) * cosd(60)        % valor esperado: 5000 (definición geométrica)

theta = acosd(dot(a, b) / (norm(a)*norm(b)))   % valor esperado: 60 (grados)
```

El mismo ejemplo, dibujado. El segmento verde es la sombra de ***b*** sobre la dirección de ***a***: mide exactamente $\boldsymbol{b} \cdot \boldsymbol{a}/\|\boldsymbol{a}\| = 50$ mm, que es lo que el producto escalar calcula sin dibujar nada.

```
a = [100; 0; 0];                      % 100 mm sobre el eje x
b = [50; 50*sqrt(3); 0];              % 100 mm, a 60 grados de a
u_a  = a / norm(a);                   % versor en la direccion de a
proy = dot(b, u_a) * u_a;             % proyeccion de b sobre a (vector)
disp(proy')                           % valor esperado: 50 0 0

figure; hold on; grid on; axis equal
quiver(0, 0, a(1), a(2), 0, 'r', 'LineWidth', 2, 'MaxHeadSize', 0.15);
quiver(0, 0, b(1), b(2), 0, 'b', 'LineWidth', 2, 'MaxHeadSize', 0.15);
plot([b(1) proy(1)], [b(2) proy(2)], 'k--');          % caida perpendicular
plot([0 proy(1)], [0 proy(2)], 'g-', 'LineWidth', 4);     % sombra de b sobre a
t = linspace(0, 60, 30);
plot(25*cosd(t), 25*sind(t), 'k-');                   % arco del angulo
```

```
text(102, 4, 'a');  text(52, 90, 'b');
text(28, 18, 'theta = 60');
text(10, -10, 'proyeccion = b . a/|a| = 50 mm');
axis([-10 120 -20 100]); xlabel('x [mm]'); ylabel('y [mm]')
title('Producto escalar como proyeccion')
```

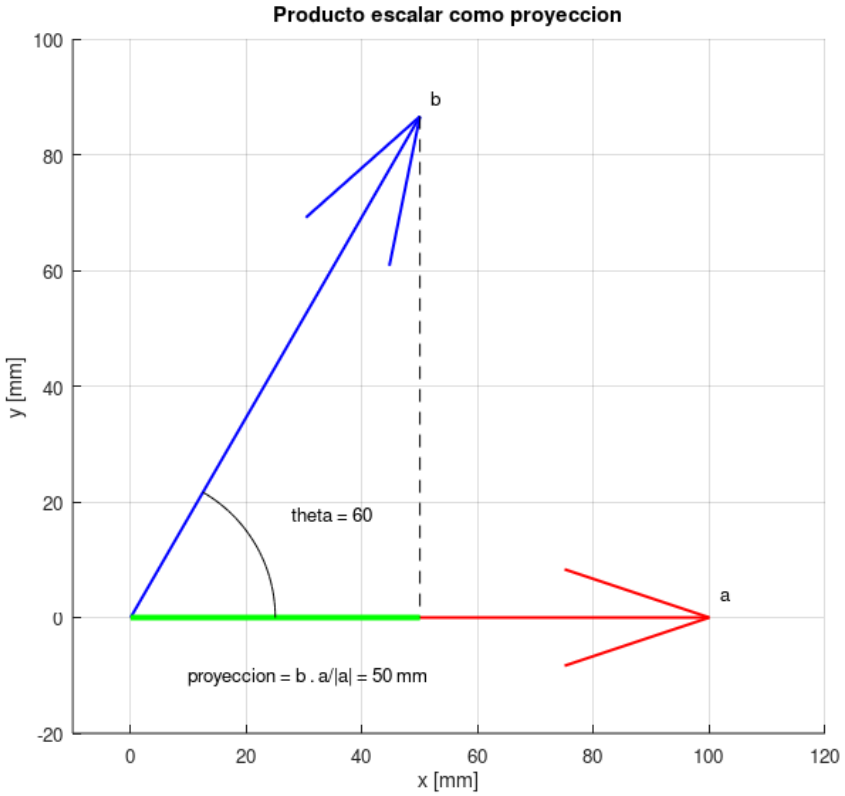


*Figura 1.13: Salida del código anterior: **a** en rojo sobre el eje x, **b** en azul a 60°, el arco del ángulo, la caída perpendicular punteada desde la punta de **b** y la proyección de 50 mm en verde.*

La igualdad entre las dos definiciones es lo que hace útil al producto escalar: nos deja calcular un ángulo, que es una cantidad geométrica, con sumas y productos de coordenadas. De ahí salen tres usos que vamos a necesitar todo el tiempo.

- El primero es detectar perpendicularidad. Si $\boldsymbol{a} \cdot \boldsymbol{b} = 0$ y ninguno de los dos es nulo, los vectores son perpendiculares. Es la manera de verificar que los ejes de una terna son ortogonales.
- El segundo es proyectar. Si $\boldsymbol{u}$ es un versor, $\boldsymbol{a} \cdot \boldsymbol{u}$ es la longitud (con signo) de la sombra de $\boldsymbol{a}$ sobre la dirección de $\boldsymbol{u}$. En particular, las coordenadas de un punto no son otra cosa que las proyecciones de su vector posición sobre los versores de la base: $p_x = \boldsymbol{p} \cdot \boldsymbol{i}$, $p_y = \boldsymbol{p} \cdot \boldsymbol{j}$, $pz = \boldsymbol{p} \cdot \boldsymbol{k}$. Cuando la base sea la de un sistema girado, esta misma fórmula nos va a dar las coordenadas del punto en ese sistema.
- El tercero es medir la orientación de un vector respecto de los ejes. Los cosenos de los ángulos que forma un versor $\boldsymbol{u}$ con x, y y z se llaman cosenos directores, y son simplemente sus tres componentes:

$$\cos\alpha = \hat{u} \cdot i = u_x, \cos\beta = \hat{u} \cdot j = u_y, \cos\gamma = \hat{u} \cdot k = u_z$$

Y como el versor tiene norma uno, sus cosenos directores cumplen $\cos^2\alpha + \cos^2\beta + \cos^2\gamma = 1$. Retengamos el nombre: en la sección 1.5 vamos a ver que una matriz de rotación es, literalmente,

una tabla de cosenos directores, los de los tres ejes de un sistema girado respecto de los del sistema fijo.

*[Figura 1.14: Un versor **u** que sale del origen y los tres ángulos α, β, γ que forma con los ejes x, y, z. Sobre cada eje se marca la proyección de **u**, de longitud cosα, cosβ, cosγ.]*

```
a = [3; 4; 0];
b = [-4; 3; 5];
dot(a, b)                       % valor esperado: 0 (son perpendiculares)

u = [1; 1; 1] / norm([1; 1; 1]);   % versor de la diagonal del cubo
cosenos = u'                    % valor esperado: [0.5774 0.5774 0.5774]
angulos = acosd(u')             % valor esperado: [54.74 54.74 54.74] grados
sum(u.^2)                       % valor esperado: 1

p = [100; 50; 30];
[dot(p,[1;0;0]) dot(p,[0;1;0]) dot(p,[0;0;1])]   % valor esperado: [100 50 30]
```

Producto vectorial

El producto vectorial de $\boldsymbol{a}$ por $\boldsymbol{b}$ es otro vector, $\boldsymbol{c} = \boldsymbol{a} \times \boldsymbol{b}$, con tres propiedades que lo definen por completo:

- Dirección: perpendicular a $\boldsymbol{a}$ y a $\boldsymbol{b}$ a la vez, es decir, normal al plano que forman.
- Módulo: $\|\boldsymbol{a}\|\,\|\boldsymbol{b}\|\,\text{sen}\theta$, que es el área del paralelogramo que tiene a $\boldsymbol{a}$ y $\boldsymbol{b}$ por lados.
- Sentido: el que da la regla de la mano derecha, curvando los dedos de $\boldsymbol{a}$ hacia $\boldsymbol{b}$.

En coordenadas se calcula con el desarrollo de un determinante simbólico, cuya primera fila son los versores de la base:

$$a \times b = \begin{vmatrix} i & j & k \\ a_x & a_y & a_z \\ b_x & b_y & b_z \end{vmatrix} = \begin{pmatrix} a_y b_z - a_z b_y \\ a_z b_x - a_x b_z \\ a_x b_y - a_y b_x \end{pmatrix}$$

A diferencia del producto escalar, el vectorial no es conmutativo: $\boldsymbol{b} \times \boldsymbol{a} = -\boldsymbol{a} \times \boldsymbol{b}$. Ya lo vimos con los versores en la sección 1.2, donde $\boldsymbol{i} \times \boldsymbol{j} = \boldsymbol{k}$ pero $\boldsymbol{j} \times \boldsymbol{i} = -\boldsymbol{k}$. Y si los dos vectores son paralelos, el producto es nulo, porque el seno del ángulo entre ellos es cero.

En robótica el producto vectorial aparece por tres razones.

- La primera, que ya usamos, es construir y verificar ternas dextrógiras: conocidos dos ejes, el tercero sale de multiplicarlos vectorialmente, en el orden correcto.
- La segunda es obtener la normal a un plano, por ejemplo la de una pieza o una pizarra sobre la que el robot tiene que dibujar, a partir de dos vectores contenidos en él.
- A la tercera la vamos a encontrar en el capítulo 3: la velocidad de un punto de un cuerpo que gira con velocidad angular $\boldsymbol{\omega}$ es $\boldsymbol{\omega} \times \boldsymbol{r}$, donde $\boldsymbol{r}$ es el vector que va del eje de giro al punto. Esa fórmula es la que arma, columna por columna, el Jacobiano de un manipulador.

*[Figura 1.15: Dos vectores **a** y **b** que parten del mismo punto, el paralelogramo que forman sombreado, y el vector **a** × **b** perpendicular al plano, con la mano derecha indicando el sentido.]*

```
a = [200; 0; 0];                % dos lados de una pizarra rectangular, en mm
b = [0; 0; 150];
n = cross(a, b)                 % valor esperado: [0; -30000; 0]
area = norm(n)                  % valor esperado: 30000 (mm^2)
n_hat = n / norm(n)             % valor esperado: [0; -1; 0], normal a la pizarra
```

BORRADOR

```
cross(b, a)                    % valor esperado: [0; 30000; 0] (sentido opuesto)
[dot(n, a) dot(n, b)]          % valor esperado: [0 0] (perpendicular a ambos)
```

El mismo ejemplo, dibujado en 3D. La pizarra es el paralelogramo gris de lados ***a*** y ***b***; ***a*** × ***b*** sale perpendicular a ella, del lado que indica la mano derecha. Como su módulo es el área (30000 mm²), para dibujarla se la normaliza y se la escala a 150 mm.

```
a = [200; 0; 0];                    % lados de la pizarra, en mm
b = [0; 0; 150];
n = cross(a, b);                    % normal, de modulo 30000 (mm^2)
n_dib = 150 * n / norm(n);          % la normal se dibuja con 150 mm de largo
disp((n/norm(n))')                  % valor esperado: 0 -1 0

figure; hold on; grid on; axis equal
V = [0 0 0; a'; (a+b)'; b'];        % vertices del paralelogramo
patch('Vertices', V, 'Faces', [1 2 3; 1 3 4], ...
      'FaceColor', [0.85 0.85 0.85], 'EdgeColor', 'none');  % paralelogramo (area)
plot3(V([1:4 1],1), V([1:4 1],2), V([1:4 1],3), 'k-');      % borde
quiver3(0,0,0, a(1),a(2),a(3), 0, 'r', 'LineWidth', 2);
quiver3(0,0,0, b(1),b(2),b(3), 0, 'b', 'LineWidth', 2);
quiver3(0,0,0, n_dib(1),n_dib(2),n_dib(3), 0, 'g', 'LineWidth', 2);
text(a(1), a(2), a(3)+10, 'a');
text(b(1), b(2), b(3)+10, 'b');
text(n_dib(1)+10, n_dib(2), n_dib(3)+20, 'a x b');
xlabel('x [mm]'); ylabel('y [mm]'); zlabel('z [mm]')
title('Producto vectorial: normal a la pizarra')
view(-50, 20)
```

El paralelogramo se arma con dos triángulos (Faces = [1 2 3; 1 3 4]) en lugar de un cuadrilátero porque algunos motores gráficos de Octave solo rellenan polígonos triangulares en 3D; así el código corre igual en cualquier instalación.

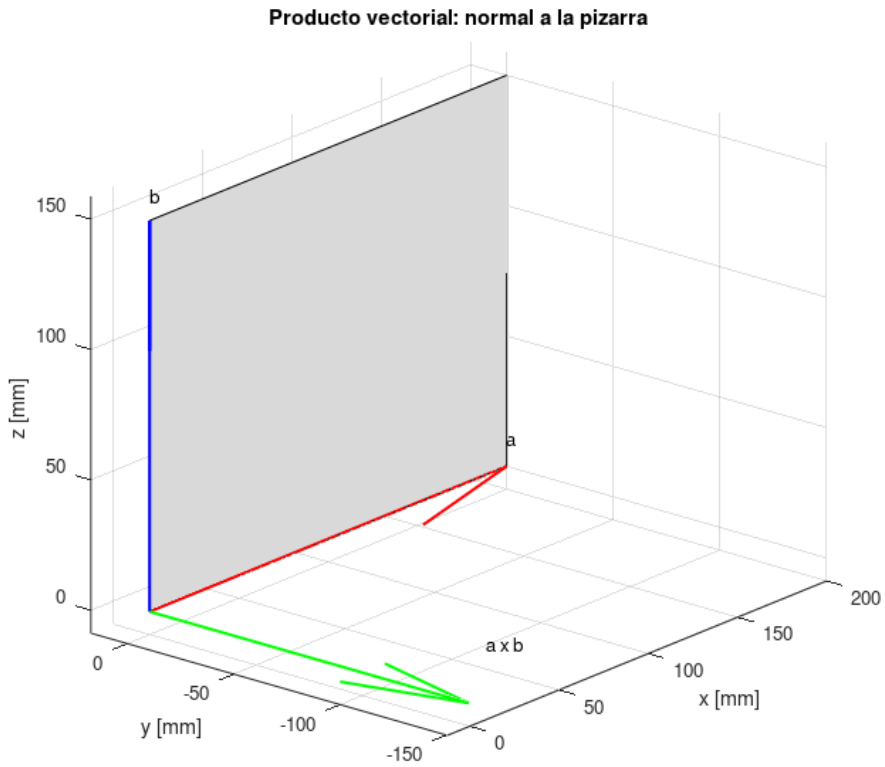


*Figura 1.16: Salida del código anterior: la pizarra gris en el plano xz, **a** en rojo sobre x, **b** en azul sobre z, y **a** × **b** en verde apuntando hacia −y.*

1.4 Traslaciones

En el espacio, los cuerpos rígidos pueden trasladarse y rotar. Describir matemáticamente la traslación es más sencillo, por lo que comenzaremos por este caso. Es el que produce una articulación prismática: un carro que corre sobre una guía, un pistón que se extiende, el eje vertical de un robot SCARA que baja la herramienta. Para describirla usamos la idea que nos va a servir también para las rotaciones: dos sistemas de referencia, uno fijo y otro que viaja con el objeto.

Dos sistemas: uno fijo y otro solidario al objeto

Supongamos dos sistemas de referencia. El primero, $\{S_0\}$, de ejes x, y, z, es fijo: lo atornillamos a la base del robot o al piso de la celda y no se mueve nunca. El segundo, $\{S_1\}$, de ejes u, v, w, es solidario a un objeto: está pegado a él y lo acompaña en cualquier movimiento, como si estuviera dibujado sobre la pieza. Al principio los dos sistemas coinciden: mismo origen, u sobre x, v sobre y y w sobre z.

Un punto P del objeto tiene coordenadas ${}^1\boldsymbol{p}$ en $\{S_1\}$. Como el sistema viaja con el objeto, esas coordenadas no cambian nunca: P es siempre el mismo punto de la pieza. Lo que cambia cuando el objeto se mueve son las coordenadas ${}^0\boldsymbol{p}$ de ese mismo punto vistas desde el sistema fijo. Mientras los dos sistemas coinciden, ${}^0\boldsymbol{p} = {}^1\boldsymbol{p}$.

Trasladar el objeto

Desplazamos ahora el objeto, sin girarlo, una distancia x_0 en la dirección de x, y_0 en la de y y z_0 en la de z. El sistema $\{S_1\}$ se va con él: su origen queda en la posición ${}^0\boldsymbol{d}$ medida en $\{S_0\}$ y, como no hubo giro, sus ejes siguen paralelos a los del sistema fijo. Mirando el triángulo que forman los dos orígenes y el punto P, la relación es inmediata:

$$\square^1 p = \begin{bmatrix} p_u \\ p_v \\ p_w \end{bmatrix}, \square^0 d = \begin{bmatrix} x_0 \\ y_0 \\ z_0 \end{bmatrix} \Rightarrow \square^0 p = \square^0 d + \square^1 p = \begin{bmatrix} x_0 + p_u \\ y_0 + p_v \\ z_0 + p_w \end{bmatrix}$$

Para pasar del sistema solidario al fijo se suma la posición del origen de $\{S_1\}$; para ir al revés, se resta: ${}^1\boldsymbol{p} = {}^0\boldsymbol{p} - {}^0\boldsymbol{d}$. Notemos que el punto no se movió respecto del objeto: lo que hicimos fue expresar el mismo punto en otro sistema.

[Figura 1.17: A la izquierda, los sistemas $\{S_0\}$ (x, y, z) y $\{S_1\}$ (u, v, w) coincidentes, con el objeto y su punto P. A la derecha, el objeto trasladado junto con $\{S_1\}$; el vector ${}^0\boldsymbol{d}$ va del origen de $\{S_0\}$ al de $\{S_1\}$, y P tiene vector posición ${}^0\boldsymbol{p}$ desde un origen y ${}^1\boldsymbol{p}$ desde el otro, formando un triángulo.]

Un ejemplo numérico. El objeto se traslada 150 mm en x y 50 mm en y, y un punto P tiene coordenadas (40, 30, 20) mm en el sistema solidario. ¿Dónde está visto desde el sistema fijo?

```
d     = [150; 50; 0];          % origen de {S1} medido en {S0}, en mm
p_uvw = [40; 30; 20];          % P en {S1}: no cambia, viaja con el objeto
p_xyz = d + p_uvw              % valor esperado: [190; 80; 20] (P en {S0})
p_xyz - d                      % valor esperado: [40; 30; 20] (de vuelta en {S1})
```

Trasladar una figura

Un objeto (el contorno de una pieza, un eslabón) no es más que un conjunto de puntos, cada uno con coordenadas fijas en su sistema solidario. Trasladar el objeto es sumarle el mismo ${}^0\boldsymbol{d}$ a cada uno.

BORRADOR

Como un cuerpo rígido no se deforma, todos sus puntos se desplazan exactamente de la misma manera: misma distancia, misma dirección.

Con la convención de la sección 1.3, una figura en Octave es una matriz de tres filas con un punto por columna. Sumarle un vector columna de tres componentes a esa matriz le suma el vector a cada columna, sin necesidad de un bucle *for* o *while*: Octave "expande" el vector para que coincida con el tamaño de la matriz.

```
obj_uvw = [0 60 60  0 0;          % placa de 60 x 40 mm descrita en {S1}
           0  0 40 40 0;          % (el ultimo punto repite el primero
           0  0  0  0 0];         %  para cerrar el dibujo)
d = [150; 50; 0];                 % traslacion del objeto, en mm
obj_xyz = obj_uvw + d;            % la misma placa vista desde {S0}
disp(obj_xyz)

norm(obj_uvw(:,2) - obj_uvw(:,1)) % valor esperado: 60 (un lado, en {S1})
norm(obj_xyz(:,2) - obj_xyz(:,1)) % valor esperado: 60 (el mismo lado, en {S0})
isequal(obj_xyz - d, obj_uvw)     % valor esperado: 1 (restar d la devuelve a {S1})
```

Salida en la consola:

```
   150   210   210   150   150
    50    50    90    90    50
     0     0     0     0     0
```

Cada columna es la columna original más (150, 50, 0). La longitud del lado se conserva, que es el *sanity check* de que la operación fue un movimiento rígido.

Para verlo, dibujamos la placa en su posición inicial (en trazos, con $\{S_1\}$ todavía coincidente con $\{S_0\}$) y la placa trasladada junto con sus ejes *u* y *v*, el vector ${}^{0}\boldsymbol{d}$ y el punto *P*:

```
d       = [150; 50; 0];
obj_uvw = [0 60 60 0 0; 0 0 40 40 0; 0 0 0 0 0];
obj_xyz = obj_uvw + d;
P_xyz   = [40; 30; 0] + d;        % un punto de la placa, visto desde {S0}

figure; hold on; axis equal
plot(obj_uvw(1,:), obj_uvw(2,:), 'b--', 'LineWidth', 1.5);   % posición inicial
plot(obj_xyz(1,:), obj_xyz(2,:), 'b-',  'LineWidth', 2);     % trasladada
L = 80;                                                      % largo de los ejes
quiver(0, 0, L, 0, 0, 'k', 'LineWidth', 1.5, 'MaxHeadSize', 0.1); text(L+3, 0,
'x');
quiver(0, 0, 0, L, 0, 'k', 'LineWidth', 1.5, 'MaxHeadSize', 0.1); text(0, L+6,
'y');
quiver(d(1), d(2), L, 0, 0, 'r', 'LineWidth', 1.5, 'MaxHeadSize', 0.1);
text(d(1)+L+3, d(2), 'u');
quiver(d(1), d(2), 0, L, 0, 'g', 'LineWidth', 1.5, 'MaxHeadSize', 0.1); text(d(1),
d(2)+L+6, 'v');
quiver(0, 0, d(1), d(2), 0, 'k--', 'MaxHeadSize', 0.1); text(d(1)/2, d(2)/2-8,
'd');
plot(P_xyz(1), P_xyz(2), 'mo', 'MarkerFaceColor', 'm'); text(P_xyz(1)+4,
P_xyz(2)+6, 'P');
axis([-20 260 -20 150]); xlabel('x [mm]'); ylabel('y [mm]')
title('Traslación del sistema {S1} solidario al objeto')
```

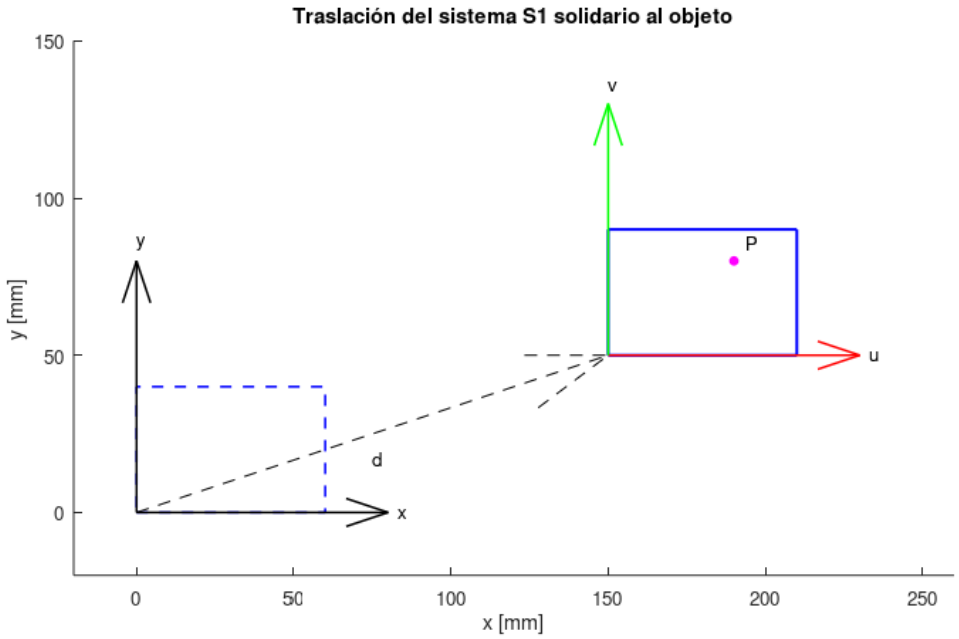


Figura 1.18: Salida del código anterior: la placa en trazos con un vértice en el origen de $\{S_0\}$, la placa trasladada con su sistema $\{S_1\}$ (ejes u en rojo y v en verde) con origen en (150, 50), el vector ${}^0\boldsymbol{d}$ en trazos negros y el punto P en (190, 80).

La otra lectura: mover el punto

Hasta acá el punto quedó quieto respecto del objeto y lo expresamos en dos sistemas distintos. La misma suma admite otra lectura: nos quedamos con un único sistema, el fijo, y lo que se mueve es el punto. Trasladar un punto es desplazarlo una cierta distancia en una cierta dirección, sin girar nada; y "una cierta distancia en una cierta dirección" es exactamente lo que describe un vector libre.

Si el punto *P* tiene vector posición $\boldsymbol{p}$ y lo trasladamos según el vector $\boldsymbol{v}$, el punto trasladado *P′* tiene vector posición $\boldsymbol{p}' = \boldsymbol{p} + \boldsymbol{v}$. En coordenadas, se suman las componentes homólogas:

$$p' = p + v = \begin{bmatrix} p_x \\ p_y \\ p_z \end{bmatrix} + \begin{bmatrix} v_x \\ v_y \\ v_z \end{bmatrix} = \begin{bmatrix} p_x + v_x \\ p_y + v_y \\ p_z + v_z \end{bmatrix}$$

*[Figura 1.19: Un punto P con su vector posición **p**, el vector de traslación **v** aplicado en P y el punto trasladado P′ con su vector posición **p′** = **p** + **v**, formando el triángulo de la suma.]*

La traslación inversa, la que devuelve *P′* a su lugar, es trasladar según $-\boldsymbol{v}$. Y dos traslaciones sucesivas, primero $\boldsymbol{v}_1$ y después $\boldsymbol{v}_2$, equivalen a una sola según $\boldsymbol{v}_1 + \boldsymbol{v}_2$. Como la suma de vectores es conmutativa, el orden no importa: da lo mismo correr el carro 100 mm en *x* y después 50 mm en *y* que hacerlo al revés. Esta es la propiedad que vamos a perder cuando entren en juego las rotaciones.

```
p = [200; 30; 0];                % un punto, en mm
v = [120; 90; 0];                % traslacion
p_t = p + v                      % valor esperado: [320; 120; 0]
```

```
p_t - v                          % valor esperado: [200; 30; 0] (vuelve a p)

v1 = [100; 0; 0]; v2 = [0; 50; 0];
isequal(p + v1 + v2, p + v2 + v1)   % valor esperado: 1 (el orden no importa)
```

Para ver la traslación, dibujamos el punto original, el trasladado y el vector **v** que los une. En línea de trazos, los dos vectores posición: se ve el triángulo de la suma $\boldsymbol{p}' = \boldsymbol{p} + \boldsymbol{v}$.

```
p   = [200; 30; 0];              % punto original, en mm
v   = [120; 90; 0];              % traslacion
p_t = p + v;                     % punto trasladado

figure; hold on; grid on; axis equal
plot(p(1),   p(2),   'bo', 'MarkerFaceColor', 'b', 'MarkerSize', 8);       % P
plot(p_t(1), p_t(2), 'ro', 'MarkerFaceColor', 'r', 'MarkerSize', 8);       % P'
quiver(p(1), p(2), v(1), v(2), 0, 'k', 'LineWidth', 1.5, 'MaxHeadSize', 0.15); % v, de P a P'
quiver(0, 0, p(1),   p(2),   0, 'b--', 'MaxHeadSize', 0.1);               % vector posicion p
quiver(0, 0, p_t(1), p_t(2), 0, 'r--', 'MaxHeadSize', 0.1);               % vector posicion p'
text(p(1)+5,   p(2)-12,  'P (200, 30)');
text(p_t(1)+5, p_t(2)+10, 'P'' (320, 120)');
text(p(1)+80,  p(2)+25,  'v = (120, 90)');
axis([-20 380 -20 160]); xlabel('x [mm]'); ylabel('y [mm]')
title('Traslacion de un punto: p'' = p + v')
```

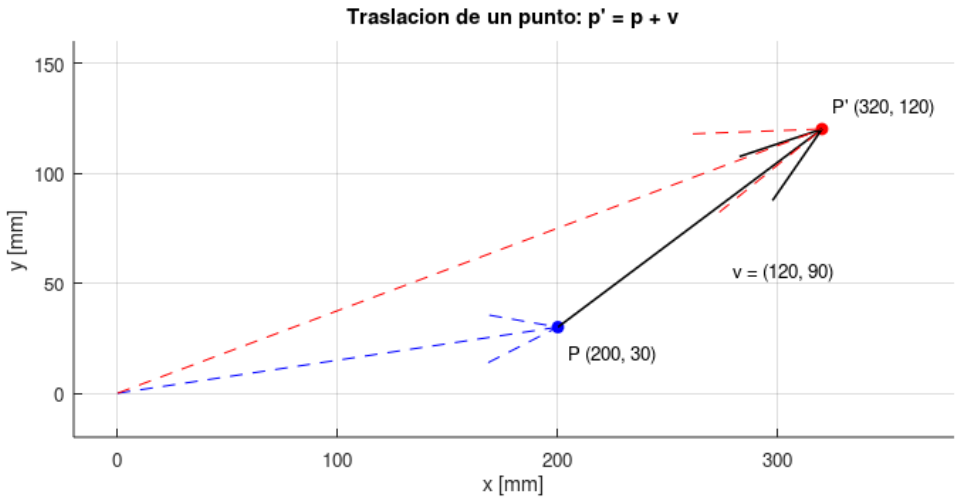


*Figura 1.20: Salida del código anterior: el punto P en azul en (200, 30), el punto trasladado P′ en rojo en (320, 120), el vector **v** en negro de P a P′ y, en trazos, los vectores posición desde el origen.*

La cuenta es la misma en los dos casos, pero el significado es distinto. Leída como cambio de sistema, ${}^{0}\boldsymbol{p} = {}^{0}\boldsymbol{d} + {}^{1}\boldsymbol{p}$ dice "el punto no se movió respecto del objeto; sus coordenadas en $\{S_1\}$ se pasan a $\{S_0\}$ sumando la posición del origen de $\{S_1\}$". Leída como movimiento, $\boldsymbol{p}' = \boldsymbol{p} + \boldsymbol{v}$ dice "el punto se corrió **v** dentro de un único sistema". En robótica la primera lectura es la que más aparece: una cámara

montada sobre el robot ve una pieza en su propio sistema y, para agarrarla, hay que expresar esa misma pieza, que no se movió, en el sistema de la base. Confundir una lectura con la otra es la fuente de buena parte de los errores de signo. Las dos nos van a acompañar con las rotaciones y con la matriz homogénea, y en cada caso vamos a decir explícitamente cuál estamos usando.

BORRADOR

Intencionalmente en blanco

BORRADOR

1.5 Rotaciones

Rotaciones en el plano

Lo que la suma no resuelve: la necesidad de un operador único

Si los robots solo tuvieran articulaciones prismáticas, la suma de vectores alcanzaría para todo: un robot cartesiano de tres ejes (como una impresora 3D, por ejemplo) se describe sumando tres desplazamientos y listo. El problema aparece cuando entran las rotaciones, que son la mayoría de las articulaciones de un robot industrial.

Vamos a ver en esta sección que una rotación no se escribe como una suma sino como un producto: el punto rotado es una matriz R multiplicada por el punto original, $\boldsymbol{p}' = R\,\boldsymbol{p}$. Un movimiento rígido general, que gira y desplaza, queda entonces como una mezcla de las dos operaciones:

$$p' = R\,p + v$$

Para un solo movimiento la mezcla se tolera. El problema es encadenarlos, que es justamente lo que hace un robot: cada eslabón está girado y desplazado respecto del anterior. Con solo dos eslabones, expresar en la base un punto dado en el sistema del segundo eslabón ya obliga a anidar las operaciones:

$$❑^{0}p = R_1\left(R_2 ❑^{2}p + d_2\right) + d_1 = R_1 R_2 ❑^{2}p + R_1 d_2 + d_1$$

Con seis eslabones la expresión se vuelve inmanejable: cada traslación queda afectada por todas las rotaciones anteriores, los términos se multiplican, y no hay manera de guardar “el movimiento del eslabón 3 respecto del 2” como un único objeto que se pueda componer con los demás. Tampoco hay una inversa simple: deshacer $\boldsymbol{p}' = R\,\boldsymbol{p} + \boldsymbol{v}$ obliga a despejar en dos pasos.

Lo que buscamos es un operador único que haga las dos cosas a la vez: una sola matriz que, multiplicada por un punto, lo rote y lo traslade. Si existiera, encadenar eslabones sería multiplicar matrices, invertir un movimiento sería invertir una matriz, y toda la maquinaria del álgebra lineal quedaría a nuestra disposición. Una matriz de 3×3 no puede hacerlo, porque cualquier producto $M\,\boldsymbol{p}$ deja fijo el origen (M por el vector nulo da el vector nulo), y una traslación mueve el origen. El truco que lo resuelve es agregar una cuarta coordenada a los puntos: las coordenadas homogéneas de la sección 1.6. Antes, necesitamos entender bien la otra mitad del problema, las rotaciones, que es lo que hacemos a continuación.

Dos sistemas con el mismo origen

Retomamos la idea de la sección 1.4. Supongamos dos sistemas de referencia con el mismo origen: $\{S_0\}$, de ejes x, y, z, fijo, y $\{S_1\}$, de ejes u, v, w, solidario a un objeto. Al principio coinciden. Ahora, en lugar de desplazar el objeto, lo giramos un ángulo θ alrededor del eje z, que sale de la hoja hacia el lector. El sistema $\{S_1\}$ gira con él: w sigue sobre z, y los ejes u y v quedan girados θ respecto de x e y, en sentido antihorario, que es el positivo según la regla de la mano derecha de la sección 1.2.

Un punto P del objeto tiene coordenadas ${}^{1}\boldsymbol{p} = (p_u, p_v)$ en $\{S_1\}$; trabajamos en el plano, porque la coordenada sobre z no cambia. Igual que en la traslación, esas coordenadas son siempre las mismas: el punto viaja con el objeto. Y la pregunta también es la misma: ¿cuánto vale ${}^{0}\boldsymbol{p}$, el mismo punto visto desde el sistema fijo?

[Figura 1.21: Los ejes x, y del sistema fijo y los ejes u, v del objeto girados θ, con origen común. El punto P con sus coordenadas p_u, p_v sobre los ejes del objeto y p^x, p_y sobre los ejes fijos. Se marcan los ángulos entre ejes: θ entre x y u, 90° + θ entre x y v, 90° − θ entre y y u, θ entre y y v.]

Proyectar sobre los ejes fijos

Llamemos $\boldsymbol{i}_x$ y $\boldsymbol{j}_y$ a los versores de los ejes *x* e *y*, e $\boldsymbol{i}_u$ y $\boldsymbol{j}_v$ a los de *u* y *v*. Por la descomposición de la sección 1.2, el vector posición de *P* es p_u veces el versor de *u* más p_v veces el versor de *v*:

$$p = p_u i_u + p_v j_v$$

Para obtener su coordenada sobre *x* proyectamos sobre ese eje, y en la sección 1.3 vimos que proyectar sobre un versor es hacer el producto escalar con él. Lo mismo para *y*:

$$\begin{matrix} p_x \\ p_y \end{matrix} \quad \begin{matrix} ¿\, p \cdot i_x = p_u (i_x \cdot i_u) + p_v (i_x \cdot j_v) \\ ¿\, p \cdot j_y = p_u (j_y \cdot i_u) + p_v (j_y \cdot j_v) \end{matrix} \Rightarrow \begin{bmatrix} p_x \\ p_y \end{bmatrix} = \begin{bmatrix} i_x \cdot i_u & i_x \cdot j_v \\ j_y \cdot i_u & j_y \cdot j_v \end{bmatrix} \begin{bmatrix} p_u \\ p_v \end{bmatrix}$$

Cada elemento de la matriz es el producto escalar entre dos versores, es decir, el coseno del ángulo que forman (sección 1.3). Mirando la figura 1.21:

$$i_x \cdot i_u = \cos\theta \,,\; i_x \cdot j_v = \cos(90^\circ + \theta) = -\sin\theta \,,\; j_y \cdot i_u = \cos(90^\circ - \theta) = \sin\theta \,,\; j_y \cdot j_v = \cos\theta$$

Reemplazando, aparece la matriz de rotación alrededor de *z*:

$$❑^0 p = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} ❑^1 p$$

¿

Acá está la diferencia de fondo con la traslación de la sección 1.4: para pasar de un sistema al otro, allá se sumaba un vector; acá se multiplica por una matriz.

Hagamos un ejemplo. Giremos un objeto 30° alrededor del eje *z*: armamos la matriz elemento por elemento con productos escalares, verificamos que sus columnas son los ejes *u* y *v*, y la aplicamos a un punto del objeto:

```
th = 30;                                % giro del objeto alrededor de z, en
grados
iu = [cosd(th); sind(th)];              % eje u del objeto, visto desde {S0}
jv = [-sind(th); cosd(th)];             % eje v del objeto, visto desde {S0}
ix = [1; 0];  jy = [0; 1];              % ejes x e y del sistema fijo

R = [dot(ix, iu)  dot(ix, jv);          % cada elemento es una proyección
     dot(jy, iu)  dot(jy, jv)]          % valor esperado: [0.8660 -0.5000; 0.5000
0.8660]
norm(R - [iu jv])                       % valor esperado: 0 (las columnas son los
ejes u y v)

p_uv = [20; 10];                        % P en {S1}, en mm: no cambia
p_xy = R * p_uv                         % valor esperado: [12.3205; 18.6603] (P
en {S0})
norm(p_xy - (p_uv(1)*iu + p_uv(2)*jv))  % valor esperado: 0 (la misma cuenta, por
componentes)
R' * p_xy                               % valor esperado: [20; 10] (de vuelta en
{S1})
```

Las columnas de R son los ejes rotados

La matriz tiene una lectura geométrica que vale más que la fórmula, y que vamos a usar en todo el libro. Su primera columna, ($\boldsymbol{i}_x \cdot \boldsymbol{i}_u$, $\boldsymbol{j}_y \cdot \boldsymbol{i}_u$) = (cosθ, senθ), contiene las proyecciones del versor $\boldsymbol{i}_u$ sobre los

ejes fijos: es el eje *u* del objeto escrito en coordenadas de $\{S_0\}$. La segunda columna, (−senθ, cosθ), es el eje *v*. En el código anterior, la línea norm(R - [iu jv]) lo confirma.

Es decir: las columnas de una matriz de rotación son los ejes del sistema solidario al objeto, escritos en coordenadas del sistema fijo. Si alguien nos da una matriz de rotación, no hace falta saber de qué ángulo salió para entender qué hace: alcanza con mirar sus columnas y dibujar hacia dónde apuntan los ejes del objeto. Y como cada elemento es el coseno del ángulo entre un eje fijo y un eje del objeto, a la matriz de rotación también se la llama matriz de cosenos directores.

[Figura 1.22: Los ejes x, y del sistema fijo y los ejes u, v del objeto girados θ. Sobre u se marcan sus componentes (cosθ, senθ) y sobre v las suyas (−senθ, cosθ), mostrando que son las columnas de R.]

Ejemplo en Octave: rotar un punto y un cuadrado en el plano

Definimos la matriz de rotación en el plano como una función anónima del ángulo (en grados, con cosd y sind) y verificamos lo que dedujimos: el punto (3, 0) girado 90°, la lectura de las columnas, la traspuesta como inversa y la conservación de las distancias. Con la convención de un punto por columna, una sola multiplicación rota la figura entera; no hace falta recorrer los puntos con un bucle.

```
R2 = @(th) [cosd(th) -sind(th);      % matriz de rotacion en el plano,
            sind(th)  cosd(th)];     % angulo en grados

p = [3; 0];
R2(90) * p                           % valor esperado: [0; 3]

R = R2(30);
disp(R)                              % columnas: los ejes u, v girados 30 grados
norm(R2(-30) - R')                   % valor esperado: 0 (R(-th) = R')
R' * R                               % valor esperado: identidad 2x2

cuad   = [0 40 40  0 0;              % cuadrado de 40 mm, un punto por columna
          0  0 40 40 0];
cuad_r = R * cuad;                   % todos los puntos rotados de una vez
disp(cuad_r)
norm(cuad_r(:,2) - cuad_r(:,1))      % valor esperado: 40 (el lado se conserva)
```

Salida en la consola (parcial):

```
   0.8660  -0.5000
   0.5000   0.8660

         0   34.6410   14.6410  -20.0000         0
         0   20.0000   54.6410   34.6410         0
```

La primera columna de *R*, (0,866; 0,5), es el eje *u* del objeto girado 30°; la segunda, (−0,5; 0,866), es el eje *v*. Donde el cálculo debería dar exactamente cero o uno, Octave puede mostrar valores del orden de 10^{-16}: son errores de redondeo de la aritmética de punto flotante, no errores de la cuenta. Por eso los controles comparan normas contra cero y no usan igualdades exactas.

Para verlo, dibujamos el cuadrado original, el rotado y los ejes del sistema girado, que son las columnas de *R*:

```
R2 = @(th) [cosd(th) -sind(th); sind(th) cosd(th)];
th = 30;  R = R2(th);
cuad   = [0 40 40  0 0;
          0  0 40 40 0];
cuad_r = R * cuad;

figure; hold on; grid on; axis equal
plot(cuad(1,:),   cuad(2,:),   'b-o', 'LineWidth', 2);          % original
plot(cuad_r(1,:), cuad_r(2,:), 'r-o', 'LineWidth', 2);          % rotado
```

```
L = 60;                                                    % largo de los
ejes dibujados
quiver(0, 0, L*R(1,1), L*R(2,1), 0, 'r--', 'MaxHeadSize', 0.1);  % eje u = 1ra
columna de R
quiver(0, 0, L*R(1,2), L*R(2,2), 0, 'g--', 'MaxHeadSize', 0.1);  % eje v = 2da
columna de R
t = linspace(0, th, 20);
plot(25*cosd(t), 25*sind(t), 'k-');                        % arco del angulo
text(27*cosd(th/2), 27*sind(th/2), 'theta = 30');
text(L*R(1,1)+2, L*R(2,1), 'u');  text(L*R(1,2)-8, L*R(2,2)+4, 'v');
legend('original', 'rotado 30 grados', 'location', 'northeast');
axis([-40 70 -10 70]); xlabel('x [mm]'); ylabel('y [mm]')
title('Rotacion de un cuadrado en el plano')
```

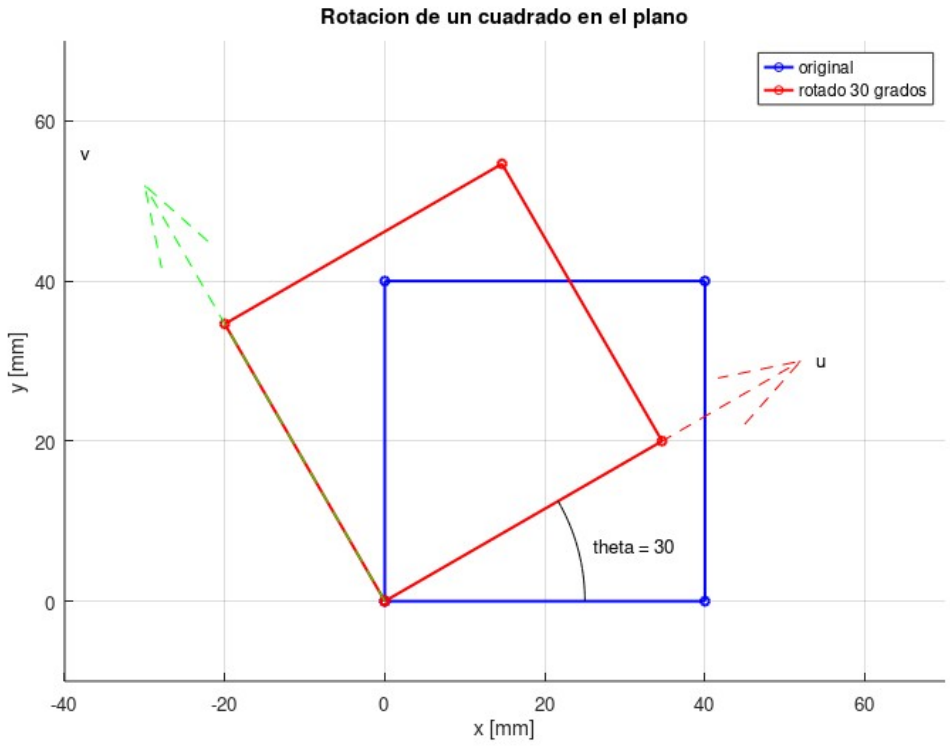


Figura 1.23: Salida del código anterior: el cuadrado original en azul, el rotado 30° en rojo, el arco del ángulo y, en trazos, los ejes u y v del sistema girado. Los lados del cuadrado rotado son paralelos a esos ejes.

Rotaciones en el espacio

Las tres matrices de rotación básicas

La rotación en el plano *xy* que acabamos de deducir es, en realidad, una rotación en el espacio alrededor del eje *z*: los puntos giran en planos paralelos a *xy* y su coordenada *z* no cambia. Para escribirla como una matriz de 3×3 basta con agregar una fila y una columna que dejen a *z* tranquila:

$$R(z,\theta)=\begin{bmatrix}\cos\theta & -\sin\theta & 0\\ \sin\theta & \cos\theta & 0\\ 0 & 0 & 1\end{bmatrix}$$

Con el mismo razonamiento se obtienen las rotaciones alrededor de los otros dos ejes. Una rotación alrededor de *x* deja fija la coordenada *x* y gira el plano *yz*; una alrededor de *y* deja fija *y* y gira el plano *zx*. Son las tres matrices de rotación básicas:

$$R(x,\alpha)=\begin{bmatrix}1 & 0 & 0\\ 0 & \cos\alpha & -\sin\alpha\\ 0 & \sin\alpha & \cos\alpha\end{bmatrix},\quad R(y,\varphi)=\begin{bmatrix}\cos\varphi & 0 & \sin\varphi\\ 0 & 1 & 0\\ -\sin\varphi & 0 & \cos\varphi\end{bmatrix}$$

[Figura 1.24: Tres dibujos lado a lado. En cada uno, un sistema xyz y el plano que gira (el yz para R(x, α), el zx para R(y, φ), el xy para R(z, θ)), con una flecha curva indicando el sentido positivo de giro según la regla de la mano derecha.]

Propiedades de las matrices de rotación

Las tres matrices básicas, y cualquier producto de ellas, comparten un conjunto de propiedades que conviene tener a mano, porque son las que vamos a usar para verificar que una matriz que escribimos es una rotación legítima.

- La primera es la que vimos en la sección 1.2 para las ternas: las columnas de R son los versores de un sistema dextrógiro, así que tienen norma uno y son perpendiculares entre sí. Si llamamos $\boldsymbol{r}_1$, $\boldsymbol{r}_2$, $\boldsymbol{r}_3$ a las columnas, el elemento (i, j) del producto $R^{\mathsf{T}}R$ es el producto escalar $\boldsymbol{r}_i \cdot \boldsymbol{r}_j$, que vale 1 si $i = j$ y 0 si no. Una matriz que cumple esto se llama ortogonal:

$$R^T R = R R^T = I \Rightarrow R^{-1} = R^T$$

La consecuencia práctica, que ya asomó en el código del plano, es que invertir una rotación es trasponerla. En el contexto de la robótica esto no es un detalle: la inversa de una matriz se calcula muchas veces por segundo en un controlador, y trasponer no cuesta nada.

- La segunda propiedad es el determinante. Toda matriz ortogonal tiene determinante +1 o −1, y las dos opciones corresponden exactamente a las dos ternas de la sección 1.2: det = +1 es una terna dextrógira, una rotación propiamente dicha; det = −1 es una terna levógira, que equivale a una rotación combinada con un reflejo en un espejo. Los reflejos no son movimientos rígidos que un robot pueda hacer, así que en este libro una matriz de rotación es una matriz 3×3 ortogonal con determinante +1.

De estas dos se deducen otras que usamos sin pensar:

- Conserva normas y ángulos: $\|R\,\boldsymbol{p}\| = \|\boldsymbol{p}\|$ y $(R\,\boldsymbol{a}) \cdot (R\,\boldsymbol{b}) = \boldsymbol{a} \cdot \boldsymbol{b}$. Es la versión algebraica de “la rotación es un movimiento rígido”.
- El producto de dos rotaciones es una rotación: si R_1 y R_2 cumplen las dos condiciones, R_1R_2 también.
- Pero el producto no es conmutativo: en general $R_1R_2 \neq R_2R_1$. Es el experimento del libro de la sección 1.1, y es el tema de la última parte de esta sección.
- La inversa de una rotación básica es la misma rotación con el ángulo cambiado de signo: $R(x, \alpha)^{\mathsf{T}} = R(x, -\alpha)$, y lo mismo para *y* y *z*.

Criterio de validación: una matriz 3×3 es una matriz de rotación válida si $R^{\mathsf{T}}R = I$ (con tolerancia numérica, por ejemplo $\|R^{\mathsf{T}}R - I\| < 10^{-9}$) y $\det(R) = +1$. Es el mismo criterio que usamos en la sección 1.2 para comprobar que una terna era dextrógira, y no es casualidad: una matriz de rotación no es otra cosa que una terna dextrógira escrita en columnas.

BORRADOR

Ejemplos en Octave: rotaciones en 3D

Octave incluye las funciones *rotx*, *roty* y *rotz* (desde la versión 7, con el ángulo en grados), pero por ahora sigamos definiendo las nuestras. Las de Octave quedan como verificación.

```
% Matrices de rotación básicas: Rx(a) = R(x,a), Ry(b) = R(y,b), Rz(g) = R(z,g);
ángulos en grados
Rx = @(a) [1 0 0; 0 cosd(a) -sind(a); 0 sind(a) cosd(a)];
Ry = @(b) [cosd(b) 0 sind(b); 0 1 0; -sind(b) 0 cosd(b)];
Rz = @(g) [cosd(g) -sind(g) 0; sind(g) cosd(g) 0; 0 0 1];

% Control de signos: cada rotacion de 90 grados lleva un eje al siguiente
Rz(90) * [3; 0; 0]              % valor esperado: [0; 3; 0] (de x a y)
Rx(90) * [0; 3; 0]              % valor esperado: [0; 0; 3] (de y a z)
Ry(90) * [0; 0; 3]              % valor esperado: [3; 0; 0] (de z a x)

% Propiedades
R = Rx(37);
norm(R'*R - eye(3))             % valor esperado: 0 (ortogonal)
det(R)                          % valor esperado: 1 (dextrogira)
norm(Rx(-37) - R')              % valor esperado: 0 (la inversa es la traspuesta)

% Verificacion contra las funciones de Octave (version 7 o posterior)
[norm(Rx(37) - rotx(37))  norm(Ry(37) - roty(37))  norm(Rz(37) - rotz(37))]   %
valor esperado: [0 0 0]
```

Las normas que deberían ser cero pueden dar valores del orden de 10^{-16}, producto del error de redondeo de la máquina.

Ahora aplicamos una rotación a distintas figuras. Tomemos un cuadrado de 40 mm y un círculo de 21 mm de radio en el plano *xy*, y los giremos 45° alrededor del eje *x*. Igual que en el plano, cada figura es una matriz con un punto por columna, y una sola multiplicación la rota entera.

```
Rx = @(a) [1 0 0; 0 cosd(a) -sind(a); 0 sind(a) cosd(a)];

cuad = [10 50 50 10 10;              % cuadrado de 40 mm en el plano xy
        10 10 50 50 10;
         0  0  0  0  0];
w    = 0:1:360;                      % circulo de 21 mm de radio en el plano xy
circ = [21*cosd(w); 21*sind(w); zeros(size(w))];

R = Rx(45);
cuad_r = R * cuad;
circ_r = R * circ;

figure; hold on; grid on; axis equal
plot3(cuad(1,:),   cuad(2,:),   cuad(3,:),   'b-',  'LineWidth', 2);
plot3(cuad_r(1,:), cuad_r(2,:), cuad_r(3,:), 'r-',  'LineWidth', 2);
plot3(circ(1,:),   circ(2,:),   circ(3,:),   'b--', 'LineWidth', 1.5);
plot3(circ_r(1,:), circ_r(2,:), circ_r(3,:), 'r--', 'LineWidth', 1.5);
quiver3(0,0,0, 60,0,0, 0, 'k', 'LineWidth', 1.5);  text(62, 0, 0, 'x');   % eje de
giro
legend('cuadrado', 'cuadrado rotado', 'circulo', 'circulo rotado', 'location',
'northeast');
xlabel('x [mm]'); ylabel('y [mm]'); zlabel('z [mm]'); view(-35, 25)
title('Rotacion de 45 grados alrededor de x')
```

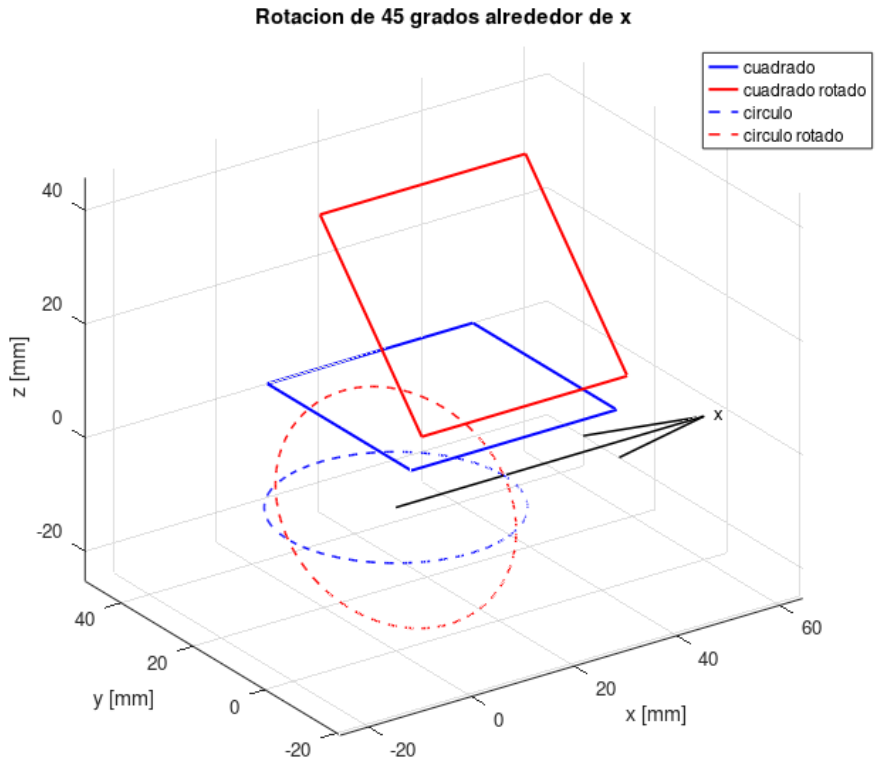


Figura 1.25: Salida del código anterior: el cuadrado y el círculo originales en azul, apoyados en el plano xy, y sus versiones rotadas en rojo, inclinadas 45°. Las figuras giran alrededor del eje x (flecha negra): los puntos que están sobre ese eje no se mueven.

El último ejemplo es más vistoso y muestra que "una figura" puede ser cualquier conjunto de puntos, incluso una imagen. Pegamos una imagen como textura sobre un plano y rotamos todos los puntos de ese plano 90° alrededor de x. Para que el ejemplo funcione sin archivos externos, la imagen se genera en el propio código (un degradado con un tablero de ajedrez); se puede reemplazar por cualquier foto con img = imread('archivo.jpg').

```
Rx = @(a) [1 0 0; 0 cosd(a) -sind(a); 0 sind(a) cosd(a)];

% Imagen de 60 x 90 pixeles generada en el codigo (o img = imread('archivo.jpg'))
[jj, ii] = meshgrid(1:90, 1:60);
img = uint8(cat(3, 255*jj/90, 255*ii/60, 255*mod(floor(jj/15) + floor(ii/15), 2)));

% Plano de 90 x 60 mm en z = 0, un punto por pixel
[X, Y] = meshgrid(linspace(0, 90, size(img,2)), linspace(0, 60, size(img,1)));
Z = zeros(size(X));

% Todos los puntos del plano en una matriz de 3 filas, rotados de una vez
P   = [X(:)'; Y(:)'; Z(:)'];
P_r = Rx(90) * P;
Xr = reshape(P_r(1,:), size(X));
Yr = reshape(P_r(2,:), size(X));
Zr = reshape(P_r(3,:), size(X));

figure; hold on; grid on; axis equal
surf(X,  Y,  Z,  'CData', img, 'FaceColor', 'texturemap', 'EdgeColor', 'none');
surf(Xr, Yr, Zr, 'CData', img, 'FaceColor', 'texturemap', 'EdgeColor', 'none');
xlabel('x [mm]'); ylabel('y [mm]'); zlabel('z [mm]'); view(-35, 25)
title('Imagen rotada 90 grados alrededor de x')
```

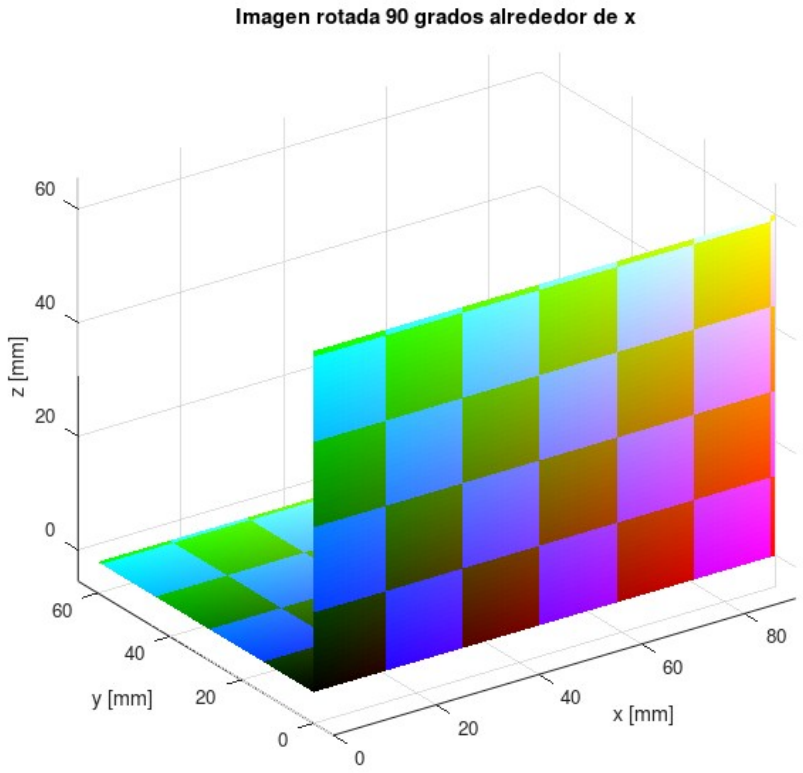


Figura 1.26: Salida del código anterior: la imagen original acostada en el plano xy y la misma imagen parada en el plano xz, después de girar 90° alrededor del eje x.

Una advertencia práctica: las texturas sobre superficies (FaceColor = 'texturemap') necesitan el motor gráfico qt o fltk, que es el que usa Octave por defecto en su interfaz gráfica. Con el motor gnuplot este último ejemplo da error; los demás funcionan con cualquiera.

Composición de rotaciones

Rotaciones sucesivas: el producto de matrices

Si a un punto le aplicamos primero una rotación R_1 y al resultado una segunda rotación R_2, el punto final es $R_2(R_1\,\boldsymbol{p})$. Como el producto de matrices es asociativo, eso es lo mismo que multiplicar el punto por la matriz $R = R_2R_1$, que representa las dos rotaciones juntas. Y con tres o más pasa lo mismo:

$$p' = R_3\,R_2\,R_1\,p$$

Hay que leerla de derecha a izquierda: la primera rotación que se aplica es la que está más cerca del punto. Una matriz de rotación compuesta se puede calcular una sola vez y aplicar después a todos los puntos que haga falta, y por las propiedades de la sección anterior sigue siendo una rotación: ortogonal y con determinante +1.

El orden importa

El producto de matrices no es conmutativo, y para las rotaciones eso tiene un significado físico muy concreto: es el experimento del libro de la sección 1.1. Hagámoslo con números. Tomamos el punto

(3, 0, 0) y le aplicamos dos rotaciones de 90°, una alrededor de *z* y otra alrededor de *x*, en los dos órdenes posibles:

- Primero *R*(*z*, 90°) y después *R*(*x*, 90°): el punto pasa de (3, 0, 0) a (0, 3, 0) y de ahí a (0, 0, 3).
- Primero *R*(*x*, 90°) y después *R*(*z*, 90°): la rotación alrededor de *x* no mueve un punto que está sobre el eje *x*, así que queda en (3, 0, 0), y la rotación alrededor de *z* lo lleva a (0, 3, 0).

Las mismas dos rotaciones, aplicadas en distinto orden, dejan el punto en lugares distintos. Cuando en un robot una articulación gira y otra gira después sobre la anterior, el orden en que se multiplican sus matrices no es un detalle de notación: es la geometría del robot.

Para verlo, repetimos en Octave el experimento del libro de la sección 1.1. Para dibujar los sistemas de ejes usamos la función dibujarEjes(*R*, ***p***, *L*, etiquetas, ancho), que viene con el material del libro en la carpeta Func_Comunes. Dibuja un sistema con orientación *R* y origen ***p***, con flechas de largo *L*: el primer eje en rojo, el segundo en verde y el tercero en azul, cada uno con su etiqueta (por ejemplo 'xyz' o 'uvw'); el ancho de línea es opcional.

La función aprovecha lo que vimos en el plano: las columnas de *R* son los ejes del sistema girado, así que cada flecha es una columna escalada al largo *L*. En el capítulo 2 vamos a usar una versión más completa, dibujarSistemasDeEjesCoordenadosDeRobot, que recibe matrices homogéneas y dibuja todos los sistemas de un robot.

El objeto es un libro de 60 × 40 × 10 mm acostado en el plano *xy*, con su centro en el origen y su sistema solidario *u*, *v*, *w* coincidente al principio con *x*, *y*, *z*. Lo ubicamos centrado a propósito: las matrices de rotación giran alrededor de ejes que pasan por el origen, así que un libro apoyado con una esquina en el origen "barrería" el espacio al girar y parecería, además, trasladado. Lo giramos 90° alrededor de *z* y 90° alrededor de *x*, en los dos órdenes, siempre respecto de los ejes fijos, y ponemos los dos resultados uno al lado del otro con subplot:

```
Rx = @(a) [1 0 0; 0 cosd(a) -sind(a); 0 sind(a) cosd(a)];   % R(x,a)
Rz = @(g) [cosd(g) -sind(g) 0; sind(g) cosd(g) 0; 0 0 1];   % R(z,g)

% Un libro de 60 x 40 x 10 mm con su centro en el origen, un vértice por columna.
% NaN corta la línea: así un solo plot3 dibuja las 12 aristas.
v = [-30  30  30 -30 -30  30  30 -30;    % los 8 vértices
     -20 -20  20  20 -20 -20  20  20;
      -5  -5  -5  -5   5   5   5   5];
orden_aristas = [1 2 3 4 1 5 6 7 8 5 NaN 2 6 NaN 3 7 NaN 4 8];
libro = NaN(3, numel(orden_aristas));
ok = ~isnan(orden_aristas);
libro(:, ok) = v(:, orden_aristas(ok));

R_zx = Rx(90) * Rz(90);                  % primero z, después x (ejes fijos)
R_xz = Rz(90) * Rx(90);                  % primero x, después z (ejes fijos)
disp(R_zx); disp(R_xz)
norm(R_zx - R_xz)                        % valor esperado: 1.7321 (distinto de 0:
no son iguales)

casos    = {R_zx, R_xz};
titulos  = {'Primero R(z,90), después R(x,90)', 'Primero R(x,90), después
R(z,90)'};
leyendas = {'Rotado z -> x', 'Rotado x -> z'};
figure
for k = 1:2
  R = casos{k};
  subplot(1, 2, k); hold on; grid on; axis equal
  h1 = plot3(libro(1,:), libro(2,:), libro(3,:), 'k--');                    %
antes de girar
  libro_r = R * libro;
```

```
  h2 = plot3(libro_r(1,:), libro_r(2,:), libro_r(3,:), 'm-', 'LineWidth', 2); %
después
  dibujarEjes(eye(3), [0;0;0], 45, 'xyz', 1);                                  %
sistema fijo
  dibujarEjes(R, [0;0;0], 30, 'uvw', 3);                                       % ejes
del libro
  axis([-50 50 -50 50 -50 50]); view(125, 20)
  xlabel('x [mm]'); ylabel('y [mm]'); zlabel('z [mm]'); title(titulos{k})
  legend([h1 h2], {'Original', leyendas{k}}, 'location', 'northeast');         %
solo los libros
end
```

Salida en la consola (las dos matrices):

```
   0  -1   0
   0   0  -1
   1   0   0

   0   0   1
   1   0   0
   0   1   0
```

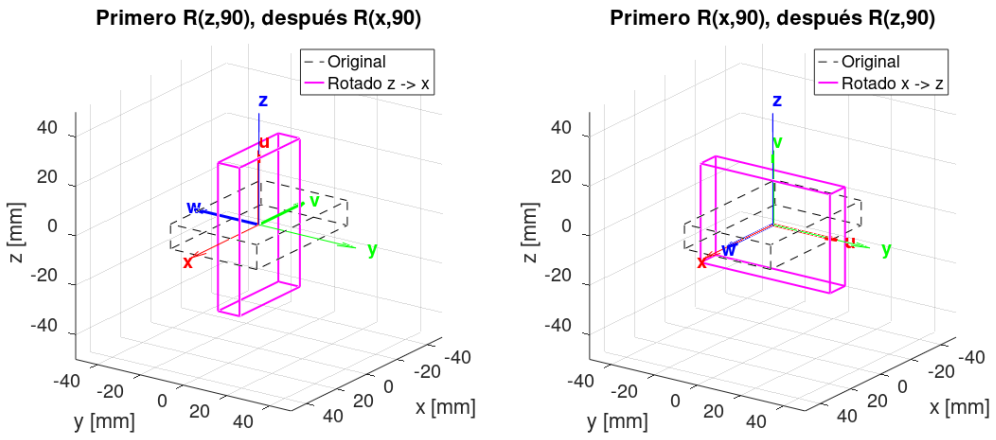


Figura 1.27: Salida del código anterior. A la izquierda, primero R(z, 90°) y después R(x, 90°): el libro queda parado sobre su lado corto, con u apuntando según z. A la derecha, primero R(x, 90°) y después R(z, 90°): el libro queda parado sobre su lado largo, con u según y y v según z. En trazos, el libro antes de girar; en líneas finas, los ejes fijos x, y, z; en gruesas, los ejes u, v, w del libro.

Leyendo las columnas de cada matriz se llega a lo mismo sin mirar el dibujo: en la primera, el eje *u* del libro (primera columna) termina en (0, 0, 1), sobre *z*; en la segunda termina en (0, 1, 0), sobre *y*. Mismas rotaciones, distinto orden, distinta orientación final.

Respecto del sistema fijo o respecto del sistema móvil

Queda una sutileza, la que más cuesta al principio y la que más vamos a usar en el capítulo 2. Cuando decimos “girar 90° alrededor de *x*” después de haber girado alrededor de *z*, ¿de qué eje *x* hablamos? Hay dos respuestas posibles:

- Alrededor del eje *x* del sistema fijo $\{S_0\}$, que no se movió.
- Alrededor del eje *x* del sistema móvil, el que viaja con el objeto y ya giró con la primera rotación.

Las dos convenciones son válidas, dan resultados distintos, y cada una tiene su regla:

- Rotaciones respecto de los ejes fijos: cada rotación nueva se premultiplica (se agrega a la izquierda). Primero R_1 y después R_2, ambas respecto de $\{S_0\}$: $R = R_2 R_1$.
- Rotaciones respecto de los ejes móviles: cada rotación nueva se posmultiplica (se agrega a la derecha). Primero R_1 y después R_2 respecto de los ejes ya girados: $R = R_1 R_2$.

¿Cuál de las dos se usa en un robot? Las dos. En una cadena cinemática algunos movimientos se describen respecto del sistema del eslabón anterior, $\{S_{i-1}\}$, y otros respecto del sistema propio, $\{S_i\}$, que ya se movió con las articulaciones anteriores. Para verlo hace falta combinar rotaciones y traslaciones en un mismo producto, así que lo retomamos en la sección 1.6 con un brazo de dos eslabones, donde cada factor se aplica sobre el sistema que dejó el anterior. Por ahora alcanza con la regla: respecto de los ejes fijos se multiplica a la izquierda; respecto de los ejes móviles, a la derecha.

BORRADOR

Intencionalmente en blanco

BORRADOR

1.6 Matriz de transformación homogénea

Coordenadas homogéneas

En la sección 1.5 dejamos planteado el problema: una rotación es un producto, $\boldsymbol{p}' = R\,\boldsymbol{p}$, pero una traslación es una suma, $\boldsymbol{p}' = \boldsymbol{p} + \boldsymbol{v}$. Son dos operaciones que, matemáticamente, se representan con elementos de distinta naturaleza. Lo que necesitamos es una sola operación que haga las dos cosas.

Matriz de transformación homogénea

Una matriz de transformación homogénea es una herramienta matemática que permite representar de forma conjunta la rotación y la traslación de un sistema de coordenadas o de un cuerpo rígido en el espacio tridimensional. Es una matriz de 4×4 formada por una submatriz de rotación R, de 3×3, y un vector de traslación $\boldsymbol{d}$, de 3×1, ordenados de la siguiente manera:

$$T = \begin{bmatrix} R_{3\times3} & d_{3\times1} \\ 0_{1\times3} & 1 \end{bmatrix}$$

La submatriz R describe una orientación, el vector $\boldsymbol{d}$ una posición, y la última fila, (0, 0, 0, 1), no lleva información: está para que la matriz sea cuadrada, posibilitando operaciones matemáticas como la multiplicación, transposición e inversa, con las reglas ordinarias del álgebra. Una matriz de transformación homogénea (MTH) no actúa sobre puntos de tres coordenadas sino sobre puntos de cuatro, escritos en coordenadas homogéneas. Eso es lo primero que vamos a ver; después volvemos a la matriz para leerla en detalle.

El truco es agregarle a cada punto una cuarta coordenada que vale siempre 1. A esa manera de escribir un punto se la llama coordenadas homogéneas:

$$p = \begin{bmatrix} p_x \\ p_y \\ p_z \end{bmatrix} \rightarrow \begin{bmatrix} p_x \\ p_y \\ p_z \\ 1 \end{bmatrix}$$

Con esa cuarta componente, la traslación de la sección 1.4 se puede escribir como el producto de una matriz de 4×4 por el punto. El vector de traslación va en la cuarta columna:

$$\begin{bmatrix} p_x + v_x \\ p_y + v_y \\ p_z + v_z \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & v_x \\ 0 & 1 & 0 & v_y \\ 0 & 0 & 1 & v_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} p_x \\ p_y \\ p_z \\ 1 \end{bmatrix}$$

Hagamos la cuenta de la primera fila: $1\cdot p_x + 0\cdot p_y + 0\cdot pz + v_x\cdot 1 = p_x + v_x$. El 1 que agregamos al punto es el que “engancha” la cuarta columna y la suma a cada componente. Las otras dos filas hacen lo mismo con y y con z, y la cuarta fila, (0, 0, 0, 1), se ocupa de que el resultado también termine en 1 y se pueda seguir multiplicando.

¿Por qué complicar las cosas si con la suma andaba bien? Porque ahora la traslación es una matriz, y una matriz se puede multiplicar por otras matrices, invertir y guardar como un único objeto. La traslación inversa, por ejemplo, es simplemente la matriz inversa, que tiene $-\boldsymbol{v}$ en la cuarta columna. Y, sobre todo, en la próxima subsección vamos a poder meter una rotación en la misma matriz.

BORRADOR

```
v  = [120; 90; 0];                  % traslación, en mm (la de la sección 1.4)
Tv = [eye(3) v; 0 0 0 1]            % matriz de traslación de 4x4
p  = [200; 30; 0; 1];               % el punto, con su 1 al final
Tv * p                              % valor esperado: [320; 120; 0; 1]
inv(Tv)                             % valor esperado: la misma matriz con -v en la
cuarta columna
inv(Tv) * (Tv * p)                  % valor esperado: [200; 30; 0; 1] (vuelve al
punto original)
```

Un error clásico es olvidarse el 1. Con un 0 en la cuarta coordenada, la cuarta columna se multiplica por cero y la traslación desaparece. Lejos de ser un defecto, esto es justo lo que necesitamos para los vectores libres de la sección 1.3: una dirección (el eje de una herramienta, una velocidad) se puede rotar, pero no tiene sentido trasladarla. Por eso, en coordenadas homogéneas, los puntos llevan un 1 al final y los vectores libres, un 0.

En computación gráfica y en visión artificial la cuarta coordenada puede valer otra cosa distinta de 1 y funciona como factor de escala, y la cuarta fila de la matriz se usa para representar perspectivas. En robótica no usamos nada de eso: los puntos terminan siempre en 1 y la cuarta fila es siempre (0, 0, 0, 1).

La matriz de transformación homogénea

Volvamos a los dos sistemas de las secciones 1.4 y 1.5: $\{S_0\}$, de ejes x, y, z, fijo, y $\{S_1\}$, de ejes u, v, w, solidario a un objeto. Ahora el objeto hace las dos cosas: gira, y su orientación queda descripta por una matriz de rotación R, y se desplaza, y su origen queda en la posición $\boldsymbol{d}$ medida en $\{S_0\}$ (el mismo ${}^0\boldsymbol{d}$ de la sección 1.4; en muchos textos a este vector se lo llama $\boldsymbol{p}$). Un punto P del objeto, de coordenadas ${}^1\boldsymbol{p}$ en el sistema solidario, se ve desde el sistema fijo en

$$❑^0 p = R ❑^1 p + d$$

que es la suma de lo que aprendimos en las dos secciones anteriores: primero las coordenadas se proyectan sobre los ejes fijos (la rotación de la 1.5) y después se les suma la posición del origen (la traslación de la 1.4). Con coordenadas homogéneas, las dos operaciones entran en una sola matriz de 4×4, la matriz de transformación homogénea:

$$\begin{bmatrix} ❑^0 p \\ 1 \end{bmatrix} = \begin{bmatrix} R_{3\times 3} & d_{3\times 1} \\ 0_{1\times 3} & 1 \end{bmatrix} \begin{bmatrix} ❑^1 p \\ 1 \end{bmatrix} \Leftrightarrow ❑^0 p = ❑^0 T_1 ❑^1 p$$

Multiplicando por bloques se recupera la expresión de arriba: el bloque R multiplica a ${}^1\boldsymbol{p}$ y el bloque $\boldsymbol{d}$ queda multiplicado por el 1. La notación es la que anunciamos en las convenciones de la sección 1.1: ${}^0T_1$ se lee “el sistema 1 respecto del 0”, y lleva coordenadas medidas en $\{S_1\}$ a coordenadas medidas en $\{S_0\}$. Cuando no haga falta aclarar los sistemas, la vamos a llamar simplemente T.

La matriz se lee por bloques, y cada bloque tiene un significado geométrico que conviene tener siempre presente:

- Las tres primeras columnas, dentro de R, son los versores de los ejes u, v, w del objeto escritos en $\{S_0\}$: la orientación de $\{S_1\}$, igual que en la sección 1.5.
- La cuarta columna, $\boldsymbol{d}$, es la posición del origen de $\{S_1\}$ medida en $\{S_0\}$: su posición.
- La cuarta fila, (0, 0, 0, 1), no lleva información: está para que el producto funcione.

Es decir, una sola matriz de 4×4 dice dónde está y cómo está orientado un sistema respecto de otro. Es exactamente la representación que la sección 1.1 nos prometía para la herramienta de un robot.

BORRADOR

Un ejemplo. El sistema $\{S_1\}$ se gira un ángulo $\alpha = 90°$ alrededor del eje x y después se traslada 75 mm sobre el eje y y −150 mm sobre el eje z. La matriz de transformación homogénea es

$$\square^0 T_1 = \begin{bmatrix} R(x,\alpha) & d \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & -1 & 75 \\ 0 & 1 & 0 & -150 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

y se lee así: el eje u sigue sobre x (primera columna), el eje v quedó apuntando según z (segunda columna), el eje w quedó apuntando según $-y$ (tercera columna), y el origen de $\{S_1\}$ está en (0, 75, −150) mm. En Octave podemos comprobarlo multiplicando la matriz por puntos fáciles de $\{S_1\}$: su origen y un punto sobre cada eje.

Una aclaración sobre el orden. Cuando armamos T poniendo R y $\boldsymbol{d}$ directamente en sus lugares, como en este ejemplo, la matriz describe una rotación seguida de una traslación: primero se gira el sistema alrededor del origen y después se lo lleva a la posición $\boldsymbol{d}$ medida en los ejes fijos. Si el orden fuera el inverso, primero trasladar y después girar alrededor de los ejes fijos, para describir el proceso matemáticamente sería necesario multiplicar dos matrices de transformación homogénea, una que describa la traslación y otra que describa la rotación. El resultado también es una matriz homogénea, pero su cuarta columna ya no es $\boldsymbol{d}$ sino $\boldsymbol{d}$ girado; lo vemos en detalle en la subsección de construcción. La forma rotación seguida de traslación es la que vamos a usar para representar cada sistema $\{S_i\}$ de la cadena cinemática respecto del anterior, $\{S_{i-1}\}$: la orientación del eslabón va en R y la posición de su origen en $\boldsymbol{d}$.

```
Rx = @(a) [1 0 0; 0 cosd(a) -sind(a); 0 sind(a) cosd(a)];   % R(x,a)
d  = [0; 75; -150];                % origen de {S1} medido en {S0}, en mm
T  = [Rx(90) d; 0 0 0 1]           % MTH de {S1} respecto de {S0}

T * [0; 0; 0; 1]                   % valor esperado: [0; 75; -150; 1] (el origen de
{S1} es d)
T * [100; 0; 0; 1]                 % valor esperado: [100; 75; -150; 1] (100 mm
sobre u = sobre x)
T * [0; 100; 0; 1]                 % valor esperado: [0; 75; -50; 1] (100 mm sobre
v = sobre z)

r = [10; 20; 30];                  % un punto cualquiera de {S1}
norm(T*[r; 1] - [Rx(90)*r + d; 1]) % valor esperado: 0 (T hace lo mismo que R*r +
d)
```

Para verlo, dibujamos los dos sistemas con dibujarEjes, leyendo R y $\boldsymbol{d}$ directamente de la matriz:

```
Rx = @(a) [1 0 0; 0 cosd(a) -sind(a); 0 sind(a) cosd(a)];
T  = [Rx(90) [0; 75; -150]; 0 0 0 1];
R  = T(1:3,1:3);  d = T(1:3,4);                % orientación y posición de {S1}

figure; hold on; grid on; axis equal
dibujarEjes(eye(3), [0;0;0], 100, 'xyz', 1);    % {S0}
dibujarEjes(R, d, 60, 'uvw', 3);                % {S1}
plot3([0 d(1)], [0 d(2)], [0 d(3)], 'k--');     % del origen de {S0} al de {S1}
xlabel('x [mm]'); ylabel('y [mm]'); zlabel('z [mm]'); view(130, 20)
title('Sistema {S1}: R(x,90) y d = (0, 75, -150) mm')
```

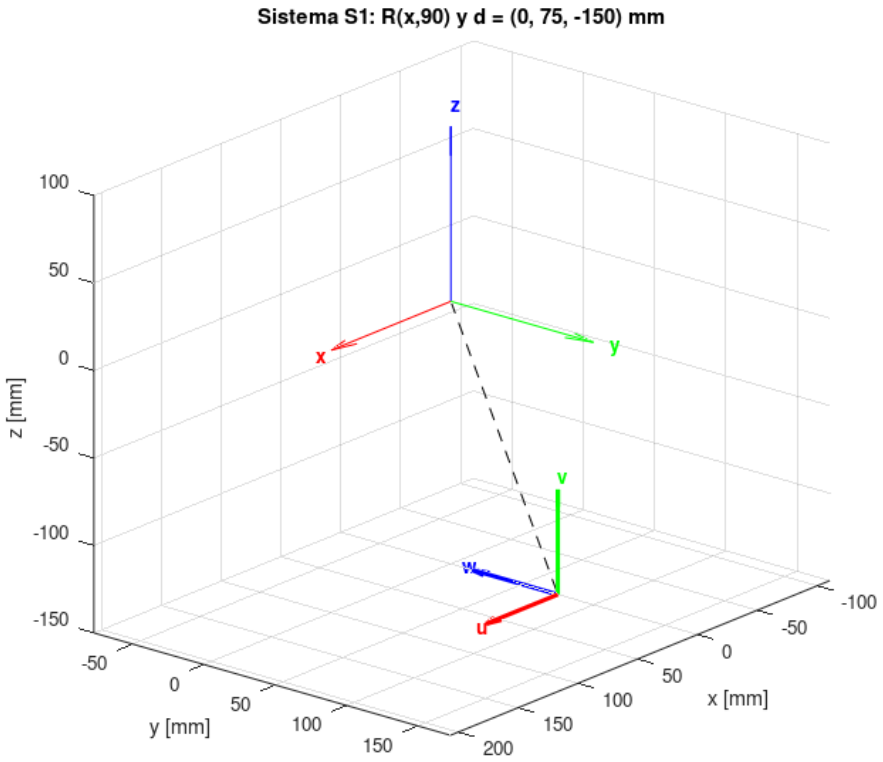


Figura 1.28: Salida del código anterior: el sistema fijo $\{S_0\}$ con ejes finos en el origen y el sistema $\{S_1\}$ con ejes gruesos en (0, 75, −150) mm, con u paralelo a x, v paralelo a z y w paralelo a −y. La línea de trazos une los dos orígenes: es la cuarta columna de T.

¿Qué representa una matriz de transformación homogénea?

La misma matriz admite tres lecturas, y conviene tenerlas separadas porque en distintos momentos del libro vamos a usar distintas:

1. Una postura. ${}^{i-1}T_i$ describe la posición y la orientación del sistema $\{S_i\}$, girado y trasladado, respecto del sistema $\{S_{i-1}\}$: sus columnas dicen hacia dónde apuntan los ejes y dónde está el origen. Es la lectura que usamos para describir dónde está la herramienta de un robot.
2. Un cambio de coordenadas. ${}^{i-1}T_i$ transforma las coordenadas de un punto medidas en $\{S_i\}$ en las coordenadas del mismo punto medidas en $\{S_{i-1}\}$. El punto no se mueve; cambia el sistema desde el que lo describimos. Es la lectura pasiva de las secciones 1.4 y 1.5, y la que vamos a usar para encadenar los eslabones de un robot.
3. Un movimiento. T rota y traslada un punto dentro de un único sistema: $\boldsymbol{p}' = T\,\boldsymbol{p}$, con todo medido en $\{S_{i-1}\}$. Es la lectura activa, la que usamos cuando movemos una figura.

Las tres lecturas comparten la misma cuenta. Lo que cambia es qué significa el resultado, y por eso vale la pena decir siempre, al escribir una T, cuál de las tres estamos usando.

Los ejes en la mano: sistemas de ejes coordenados físicos

Todo lo que venimos haciendo con matrices se entiende mucho mejor si se lo puede ver en la realidad en vez de en papel, la pantalla, o la imaginación. Para eso usamos un recurso muy simple: un sistema de ejes coordenados físico, construido con un cubo de 30 mm impreso en 3D que tiene tres agujeros hexagonales en tres caras perpendiculares, donde se encastran tres lapiceras de colores que

hacen de ejes. La roja marca el eje *x*, la verde el eje *y* y la azul el eje *z*, los mismos colores que usa dibujarEjes en todos los gráficos del libro. El archivo para imprimir el cubo, Cubo milimetros.stl, viene con el material del libro.

Armados dos sistemas de ejes coordenados físicos, uno queda quieto sobre la mesa y hace de $\{S_0\}$; el otro se mueve con la mano y hace de $\{S_1\}$. Antes de usarlos conviene hacer el control de la sección 1.2: con la mano derecha, los dedos curvados del eje *x* (rojo) hacia el eje *y* (verde), el pulgar tiene que apuntar como el eje *z* (azul). Si no, la terna quedó levógira y hay que cambiar una lapicera de agujero.

[Figura 1.29: Foto de los dos sistemas de ejes coordenados físicos: a la izquierda $\{S_0\}$ apoyado sobre la mesa, a la derecha $\{S_1\}$ girado y desplazado.]

Algunos ejercicios para hacer con los sistemas físicos, con la matriz al lado:

1. Leer una matriz de rotación. Se gira $\{S_1\}$ a mano y se anota hacia dónde apunta cada uno de sus ejes en términos de los de $\{S_0\}$: "el eje *x* (rojo) apunta como el eje *y* (verde) fijo", "el eje *y* (verde) apunta como el eje *z* (azul) fijo", y así. Esas tres respuestas, escritas como vectores, son las tres columnas de *R*. Después se comprueba en Octave con dibujarEjes.
2. El orden importa. Se repite con el sistema físico móvil el ejemplo del libro de la sección 1.5: girar 90° alrededor de *z* y después 90° alrededor de *x*, y después al revés, siempre alrededor de los ejes del sistema fijo. El sistema móvil termina en dos orientaciones distintas, que coinciden con las dos matrices del ejemplo.
3. Ejes fijos o ejes móviles. Se hacen los mismos dos giros, pero el segundo alrededor del eje del propio sistema móvil y no del eje del fijo. Se ve con las manos por qué en un caso se premultiplica y en el otro se posmultiplica.
4. Construir una matriz homogénea. Se reproduce el ejemplo del punto *K* de la subsección siguiente: se gira $\{S_1\}$ 90° alrededor de *z* y se lo lleva con una regla a 100 mm sobre *x*, 200 mm sobre *y* y 300 mm sobre *z* (o a una escala más cómoda para la mesa). Después se ubica el punto *K* midiendo sobre los ejes de $\{S_1\}$ y se comparan sus coordenadas, medidas sobre los de $\{S_0\}$, con las que da la matriz.

Los sistemas de ejes coordenados físicos nos van a acompañar en el capítulo 2: el algoritmo de Denavit y Hartenberg se reduce a llevar un sistema sobre el siguiente con dos giros y dos traslaciones, y hacerlo con dos sistemas físicos en la mano es la mejor manera de entender qué significa cada uno de sus parámetros.

Construcción de la matriz: el orden importa

Para construir una matriz homogénea lo primero es entender aquello que se pretende modelar. Siempre suponemos que, al principio, el sistema fijo y el móvil coinciden, y después se aplican al móvil, en orden, los giros y traslaciones que indica el enunciado. Cada operación es una matriz homogénea básica: una rotación pura, con $\boldsymbol{d} = 0$, o una traslación pura, con $R = I$:

$$T_{rot} = \begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix}, T_{tras} = \begin{bmatrix} I & d \\ 0 & 1 \end{bmatrix}$$

Y se componen con la regla de la sección 1.5: si las operaciones se hacen respecto de los ejes fijos, cada operación nueva se premultiplica (se agrega a la izquierda). Con dos operaciones hay dos órdenes posibles, y no dan lo mismo:

$$T_{tras}T_{rot}=\begin{bmatrix}R & d\\ 0 & 1\end{bmatrix},T_{rot}T_{tras}=\begin{bmatrix}R & Rd\\ 0 & 1\end{bmatrix}$$

La orientación final es la misma en los dos casos, pero la posición no: si primero se traslada y después se rota, la rotación alrededor de un eje del sistema fijo se lleva consigo el origen ya desplazado, y el vector de traslación queda rotado. Es la versión homogénea de lo que vimos con el libro en la sección 1.5. La forma $T = [R\ \mathbf{d};\ 0\ 1]$ que usamos en la subsección anterior corresponde, entonces, a "primero rotar, después trasladar".

Un ejemplo, para hacerlo primero a ojo y después con la matriz. El sistema $\{S_1\}$ se gira 90° alrededor del eje z y después se traslada según $\mathbf{d}$ = (100, 200, 300) mm. ¿Dónde está, en $\{S_0\}$, el punto K de coordenadas (100, 100, 100) mm en $\{S_1\}$? Después del giro, el eje u quedó sobre y, el eje v sobre $-x$ y el eje w sigue sobre z. Entonces, partiendo del origen de $\{S_1\}$, que está en (100, 200, 300): los 100 mm sobre u suman 100 en y, los 100 mm sobre v restan 100 en x y los 100 mm sobre w suman 100 en z. El punto queda en (0, 300, 400). La matriz llega a lo mismo sin dibujar nada, y también muestra qué pasa con el orden inverso:

```
Rz = @(g) [cosd(g) -sind(g) 0; sind(g) cosd(g) 0; 0 0 1];   % R(z,g)
T_rot  = [Rz(90) [0; 0; 0]; 0 0 0 1];          % giro de 90 grados alrededor de z
T_tras = [eye(3) [100; 200; 300]; 0 0 0 1];    % traslación, en mm
k = [100; 100; 100; 1];                        % el punto K en {S1}

T_rt = T_tras * T_rot                          % primero rotar, después trasladar
T_rt * k                                       % valor esperado: [0; 300; 400; 1]

T_tr = T_rot * T_tras                          % primero trasladar, después rotar
T_tr * k                                       % valor esperado: [-300; 200; 400;
1]
```

Las dos matrices tienen el mismo bloque de rotación y distinta cuarta columna: (100, 200, 300) en el primer caso y (−200, 100, 300), que es $\mathbf{d}$ girado 90°, en el segundo.

Con más de una rotación el procedimiento es el mismo: se multiplican las rotaciones en el orden que corresponde y se agrega la traslación. Si $\{S_1\}$ se gira un ángulo α = 90° alrededor de x, después un ángulo θ = 90° alrededor de z (los dos ejes del sistema fijo) y por último se traslada 75 mm sobre y y −50 mm sobre z, la matriz es

$$\prescript{0}{}{T}_1=\begin{bmatrix}R(z,\theta)R(x,\alpha) & d\\ 0 & 1\end{bmatrix}=\begin{bmatrix}0 & 0 & 1 & 0\\ 1 & 0 & 0 & 75\\ 0 & 1 & 0 & -50\\ 0 & 0 & 0 & 1\end{bmatrix}$$

El problema típico de aplicación es el mismo que resolvimos con el punto K: conocemos las coordenadas de un punto en el sistema móvil y queremos las del sistema fijo. Por ejemplo, el vector $\mathbf{r}$ tiene coordenadas (−20, 70, 30) mm en $\{S_1\}$, y $\{S_1\}$ fue girado α = 90° alrededor de x, después φ = −90° alrededor de y y trasladado (60, −30, 80) mm. Ese "no olvidar el 1" que comentamos al principio aparece acá en el código:

```
Rx = @(a) [1 0 0; 0 cosd(a) -sind(a); 0 sind(a) cosd(a)];   % R(x,a)
Ry = @(b) [cosd(b) 0 sind(b); 0 1 0; -sind(b) 0 cosd(b)];   % R(y,b)
Rz = @(g) [cosd(g) -sind(g) 0; sind(g) cosd(g) 0; 0 0 1];   % R(z,g)

T = [Rz(90)*Rx(90) [0; 75; -50]; 0 0 0 1]      % valor esperado: la matriz de
arriba

r_uvw = [-20; 70; 30; 1];                      % no olvidar el 1
```

```
T = [Ry(-90)*Rx(90) [60; -30; 80]; 0 0 0 1];
r_xyz = T * r_uvw                              % valor esperado: [-10; -60; 60; 1]
```

Una advertencia que vale para todo lo que sigue. En los ejemplos de esta subsección primero rotamos y después trasladamos, y siempre respecto de los ejes fijos. En un robot suele ser al revés: cada eslabón se traslada a lo largo de su propio eje y después gira la articulación siguiente, y esas operaciones están referidas a los ejes del eslabón, que se mueven. La matriz que resulta es otra, y es el tema de la próxima subsección.

Sucesión de transformaciones: un brazo de dos eslabones

Podemos usar sucesiones de traslaciones y rotaciones para encontrar la posición y la orientación del extremo de un robot. Tomemos el brazo más simple que tiene interés: dos eslabones en el plano *xy*, de largos L_1 y L_2, unidos por dos articulaciones de rotación alrededor de *z*, con ángulos q_1 y q_2. Queremos la postura de la punta respecto de la base.

Describamos lo que hace el brazo, desde la base hacia la punta, como una serie de movimientos de un sistema de ejes que viaja por la cadena:

1. Gira q_1 alrededor de *z*: es la primera articulación.
2. Avanza L_1 a lo largo de su eje *x*, que ya no es el *x* de la base sino el que quedó apuntando a lo largo del primer eslabón. Así llega al codo.
3. Gira q_2 alrededor de su eje *z*: es la segunda articulación.
4. Avanza L_2 a lo largo de su nuevo eje *x*, a lo largo del segundo eslabón. Así llega a la punta.

Acá está la diferencia con la subsección anterior: cada movimiento se hace respecto de los ejes que dejó el movimiento anterior, no respecto de los ejes fijos. Algunos se describen en el sistema de un eslabón y otros en el del siguiente. Es el caso de ejes móviles de la sección 1.5, y la regla es la misma: cada matriz nueva se posmultiplica (se agrega a la derecha). La postura de la punta queda

$$T = R(z, q_1)\,T_x(L_1)\,R(z, q_2)\,T_x(L_2)$$

donde cada $R(z, q)$ es una rotación homogénea pura y cada $T_x(L)$ una traslación pura de L sobre *x*. El producto se lee de izquierda a derecha si pensamos en ejes que se mueven, y de derecha a izquierda si pensamos en ejes fijos: las dos lecturas dan la misma matriz.

Con $L_1 = L_2 = 300$ mm, $q_1 = 30°$ y $q_2 = 60°$, la cuarta columna de T tiene que coincidir con la cuenta trigonométrica que se hace a mano para este brazo, $x = L_1 \cos q_1 + L_2 \cos(q_1 + q_2)$ e $y = L_1 \operatorname{sen} q_1 + L_2 \operatorname{sen}(q_1 + q_2)$:

```
Rz = @(g) [cosd(g) -sind(g) 0; sind(g) cosd(g) 0; 0 0 1];   % R(z,g)
H  = @(R, d) [R d; 0 0 0 1];                 % arma una MTH a partir de R y d

L1 = 300;  L2 = 300;                         % largos de los eslabones, en mm
q1 = 30;   q2 = 60;                          % ángulos articulares, en grados

A1 = H(Rz(q1), [0; 0; 0]);                   % gira q1 alrededor de z
A2 = H(eye(3), [L1; 0; 0]);                  % avanza L1 sobre su x
A3 = H(Rz(q2), [0; 0; 0]);                   % gira q2 alrededor de su z
A4 = H(eye(3), [L2; 0; 0]);                  % avanza L2 sobre su nuevo x

T = A1 * A2 * A3 * A4;                       % ejes móviles: de izquierda a derecha
T(1:3,4)'                                    % valor esperado: 259.8076  450.0000
0
[L1*cosd(q1) + L2*cosd(q1+q2), L1*sind(q1) + L2*sind(q1+q2)]   % valor esperado:
259.81  450.00
T(1:3,1:3)                                   % valor esperado: R(z,90), la punta
```

BORRADOR

```
giró q1 + q2

T_mal = A4 * A3 * A2 * A1;                    % el orden al revés
T_mal(1:3,4)'                                 % valor esperado: 450.0000  259.8076
0 (otro punto)
```

El bloque de rotación es $R(z, 90°)$, porque los dos giros son alrededor del mismo eje y se suman. Octave puede mostrar algún 0.0000 o −0.0000 donde debería haber un cero exacto: es el redondeo de punto flotante que ya comentamos en la sección 1.5. Y el producto en el orden inverso da otro punto, (450, 259.8): el orden de las matrices no es un detalle de notación, es la geometría del brazo.

Para verlo, dibujamos los eslabones y un sistema de ejes en la base, en el codo y en la punta. Los sistemas intermedios salen de los productos parciales: $A_1A_2A_3$ es el sistema del segundo eslabón, ubicado en el codo.

```
Rz = @(g) [cosd(g) -sind(g) 0; sind(g) cosd(g) 0; 0 0 1];
H  = @(R, d) [R d; 0 0 0 1];
L1 = 300;  L2 = 300;  q1 = 30;  q2 = 60;
A1 = H(Rz(q1), [0;0;0]);  A2 = H(eye(3), [L1;0;0]);
A3 = H(Rz(q2), [0;0;0]);  A4 = H(eye(3), [L2;0;0]);
T1 = A1 * A2 * A3;                            % sistema del eslabón 2, en el codo
T2 = T1 * A4;                                 % sistema de la punta

figure; hold on; grid on; axis equal
plot([0 T1(1,4) T2(1,4)], [0 T1(2,4) T2(2,4)], 'k-o', 'LineWidth', 3);   %
eslabones
dibujarEjes(eye(3), [0;0;0], 100, 'xyz', 1);                              % base
dibujarEjes(T1(1:3,1:3), T1(1:3,4), 80, 'xyz', 2);                        % codo
dibujarEjes(T2(1:3,1:3), T2(1:3,4), 80, 'xyz', 2);                        % punta
text(-40, -40, 'S0');  text(T1(1,4)+10, T1(2,4)-25, 'S1');  text(T2(1,4)+10,
T2(2,4)-25, 'S2');
view(0, 90); axis([-60 420 -60 560]); xlabel('x [mm]'); ylabel('y [mm]')
title('Brazo de dos eslabones: T = R(z,q1) Tx(L1) R(z,q2) Tx(L2)')
```

BORRADOR

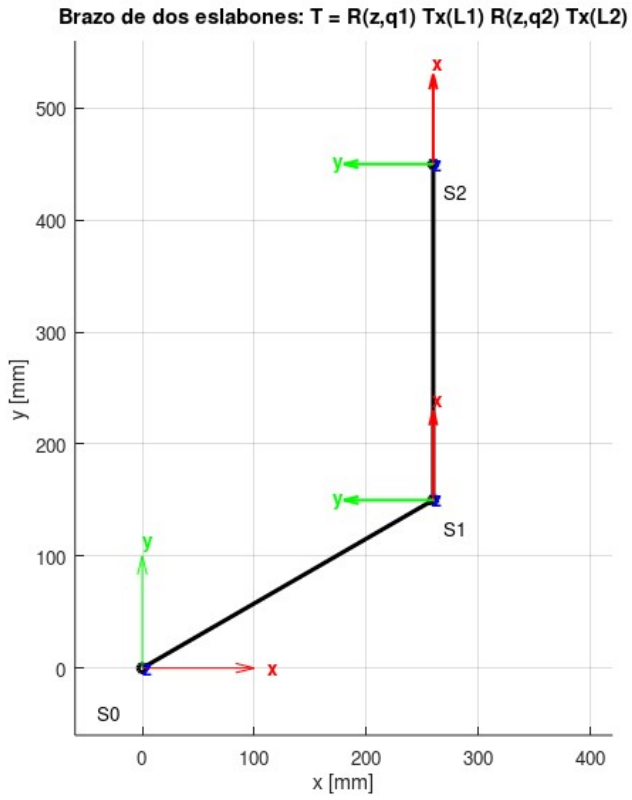


Figura 1.30: Salida del código anterior: el brazo con el primer eslabón a 30° y el segundo vertical, el sistema de la base en el origen, el del codo en (259.8, 150) y el de la punta en (259.8, 450), estos dos con su eje x apuntando a lo largo del segundo eslabón.

Este ejemplo es, en pequeño, todo el capítulo 2. Allí vamos a asignar un sistema a cada eslabón con una regla fija, la de Denavit y Hartenberg, y la postura del extremo va a ser el producto, hacia la derecha, de una matriz homogénea por cada articulación.

Aplicación a figuras: un cuadrado y un círculo

Todo lo anterior se aplica de igual manera a un punto o a una figura entera. Con la convención de la sección 1.3, una figura es una matriz de tres filas con un punto por columna; para usarla con una matriz homogénea alcanza con agregarle una cuarta fila de unos, y una sola multiplicación la rota y la traslada completa.

Tomamos un círculo de 21 mm de radio y un cuadrado de 30 mm de lado, los dos en el plano $z = 10$ mm, y les aplicamos una rotación de 45° alrededor de x y una traslación de (35, −45, 60) mm, en los dos órdenes posibles y respecto de los ejes fijos. Ponemos los dos resultados uno al lado del otro con subplot:

```
Rx = @(a) [1 0 0; 0 cosd(a) -sind(a); 0 sind(a) cosd(a)];   % R(x,a)
H  = @(R, d) [R d; 0 0 0 1];

% Figuras en el plano z = 10 mm, un punto por columna
w    = 0:1:360;
circ = [21*cosd(w); 21*sind(w); 10*ones(size(w))];            % círculo de 21 mm
de radio
cuad = [-15 15 15 -15 -15; -15 -15 15 15 -15; 10 10 10 10 10];  % cuadrado de 30 mm
de lado
```

```
Trot  = H(Rx(45), [0; 0; 0]);           % rotación de 45 grados alrededor de x
Ttras = H(eye(3), [35; -45; 60]);       % traslación, en mm
T_rt  = Ttras * Trot                    % primero rotar, después trasladar
T_tr  = Trot * Ttras                    % primero trasladar, después rotar

% Coordenadas homogéneas: se agrega una fila de unos
circ_h = [circ; ones(1, columns(circ))];
cuad_h = [cuad; ones(1, columns(cuad))];

casos   = {T_rt, T_tr};
titulos = {'Rotar y después trasladar', 'Trasladar y después rotar'};
figure
for k = 1:2
  T = casos{k};
  c = T * circ_h;   q = T * cuad_h;     % una multiplicación transforma la figura
entera
  subplot(1, 2, k); hold on; grid on; axis equal
  h1 = plot3(circ(1,:), circ(2,:), circ(3,:), 'b--');
  plot3(cuad(1,:), cuad(2,:), cuad(3,:), 'b--');
  h2 = plot3(c(1,:), c(2,:), c(3,:), 'r-', 'LineWidth', 2);
  plot3(q(1,:), q(2,:), q(3,:), 'r-', 'LineWidth', 2);
  dibujarEjes(eye(3), [0;0;0], 40, 'xyz', 1);
  axis([-30 70 -100 40 -20 90]); view(-40, 20)
  xlabel('x [mm]'); ylabel('y [mm]'); zlabel('z [mm]'); title(titulos{k})
  legend([h1 h2], {'Original', 'Transformada'}, 'location', 'southoutside');
end
```

Salida en la consola (las dos matrices):

```
T_rt =
    1.0000         0         0   35.0000
         0    0.7071   -0.7071  -45.0000
         0    0.7071    0.7071   60.0000
         0         0         0    1.0000

T_tr =
    1.0000         0         0   35.0000
         0    0.7071   -0.7071  -74.2462
         0    0.7071    0.7071   10.6066
         0         0         0    1.0000
```

El bloque de rotación es el mismo, así que las dos figuras transformadas quedan igual de inclinadas; lo que cambia es dónde quedan. En la primera matriz la cuarta columna es la traslación tal cual; en la segunda es la traslación girada 45° alrededor de *x*, porque la rotación se aplicó después y arrastró a la figura ya desplazada.

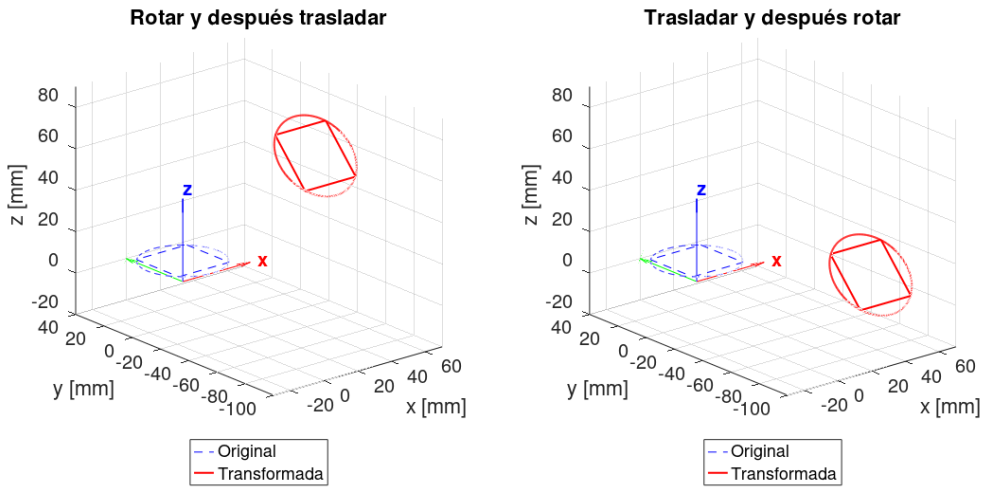


Figura 1.31: Salida del código anterior. En cada gráfico, el círculo y el cuadrado originales en trazos azules, cerca del origen, y las figuras transformadas en rojo, inclinadas 45°. A la izquierda, primero rotar y después trasladar: las figuras quedan centradas en (35, −52.1, 67.1). A la derecha, primero trasladar y después rotar: quedan más abajo y más lejos sobre −y, centradas en (35, −81.3, 17.7).

Criterios de validación

Una matriz homogénea mal armada no da indicios de error: Al fin y al cabo es solo una colección de números, y los números parecen razonables. Por eso, antes de usar una T, conviene pasarla por una lista corta de controles, todos fáciles de hacer en Octave:

1. La cuarta fila es exactamente (0, 0, 0, 1).
2. El bloque R es una rotación legítima: $R^TR = I$ con tolerancia numérica y $\det(R) = +1$. Es el criterio de la sección 1.5.
3. El origen de $\{S_1\}$ va a parar a la cuarta columna: $T\,(0, 0, 0, 1) = (\boldsymbol{d}, 1)$.
4. Las distancias se conservan: la distancia entre dos puntos de un cuerpo es la misma antes y después de transformarlos. Es el *sanity check* de la sección 1.3, y detecta cualquier matriz que estire o deforme.
5. El orden del producto coincide con el enunciado: respecto de los ejes fijos, cada operación nueva va a la izquierda; respecto de los ejes móviles, a la derecha. Este control no se puede automatizar, pero un dibujo de los sistemas con dibujarEjes, o los sistemas físicos en la mano, lo resuelven en segundos.

```
Rz = @(g) [cosd(g) -sind(g) 0; sind(g) cosd(g) 0; 0 0 1];   % R(z,g)
T = [Rz(90) [100; 200; 300]; 0 0 0 1];       % la matriz a validar
R = T(1:3,1:3);  d = T(1:3,4);

isequal(T(4,:), [0 0 0 1])                   % valor esperado: 1 (cuarta fila)
norm(R'*R - eye(3)) < 1e-9                   % valor esperado: 1 (R ortogonal)
abs(det(R) - 1) < 1e-9                       % valor esperado: 1 (det = +1,
dextrógira)
isequal(T * [0; 0; 0; 1], [d; 1])            % valor esperado: 1 (el origen va a d)
a = [10; 0; 0; 1];  b = [10; 40; 30; 1];
[norm(b - a)  norm(T*b - T*a)]               % valor esperado: 50 50 (se conservan
las distancias)

T_mal = T;  T_mal(1:3,1:3) = 2*R;            % una matriz que no es una MTH válida
Rm = T_mal(1:3,1:3);
norm(Rm'*Rm - eye(3)) < 1e-9                 % valor esperado: 0 (el control la
detecta)
```

Con esto termina el recorrido del capítulo. Empezamos con un punto y tres números, y llegamos a una matriz de 4×4 que dice dónde está y hacia dónde apunta un sistema de coordenadas, y que se encadena multiplicando. En el capítulo 2 vamos a pegarle un sistema a cada eslabón de un robot con una regla fija, y la postura de la herramienta va a salir, como en el brazo de dos eslabones, de multiplicar una matriz homogénea por cada articulación.

BORRADOR

1.7 Ejercicios

Los ejercicios están agrupados por tipo y siguen el orden del capítulo. Todas las longitudes están en milímetros y todos los ángulos en grados. Conviene resolver cada uno primero a mano, o con los sistemas de ejes coordenados físicos cuando se pueda, y después verificarlo en Octave con las funciones del capítulo. Las respuestas están al final.

Vectores (sección 1.3)

1. Los puntos P y Q tienen vectores posición $\boldsymbol{p}$ = (300, 400, 0) y $\boldsymbol{q}$ = (600, 0, 0). Calcular el vector libre que va de P a Q, la distancia entre los dos puntos y el versor de la dirección PQ.
2. Calcular el ángulo entre $\boldsymbol{a}$ = (100, 0, 0) y $\boldsymbol{b}$ = (100, 100, 0), y la longitud de la proyección de $\boldsymbol{b}$ sobre la dirección de $\boldsymbol{a}$.
3. Una pizarra rectangular tiene dos lados $\boldsymbol{a}$ = (0, 300, 0) y $\boldsymbol{b}$ = (0, 0, 200). Calcular el versor normal a la pizarra, en el sentido de $\boldsymbol{a} \times \boldsymbol{b}$, y su área.

Traslaciones (sección 1.4)

4. El sistema $\{S_1\}$ está trasladado, sin girar, de modo que su origen queda en (200, −100, 50) medido en $\{S_0\}$. Un punto tiene coordenadas (30, 40, 0) en $\{S_1\}$. ¿Cuáles son sus coordenadas en $\{S_0\}$?
5. Un punto tiene coordenadas (500, 250, 100) en $\{S_0\}$, y el origen de $\{S_1\}$, con ejes paralelos a los de $\{S_0\}$, está en (100, 100, 100). ¿Cuáles son sus coordenadas en $\{S_1\}$?

Rotaciones básicas (sección 1.5)

6. Escribir $R(y, 90°)$ y verificar con ella que una rotación positiva alrededor de y lleva el eje z sobre el eje x: aplicarla al punto (0, 0, 100).
7. El sistema $\{S_1\}$ está girado 30° alrededor de z. Un punto está sobre su eje u, a 100 mm del origen. ¿Cuáles son sus coordenadas en $\{S_0\}$?
8. Girar el punto (0, 100, 0) 90° alrededor de x. Después verificar en Octave que $R(x, 90°)^{\mathsf{T}} = R(x, -90°)$.

Composición de rotaciones: ejes fijos y móviles (sección 1.5)

9. Un objeto se gira 90° alrededor de z y después 90° alrededor de y, los dos ejes del sistema fijo. Calcular la matriz de rotación resultante y decir hacia dónde apunta cada eje del objeto.
10. Los mismos dos giros, pero el segundo alrededor del eje v del propio objeto, ya girado. Calcular la matriz y compararla con la del ejercicio 9.
11. Un objeto se gira, siempre respecto de los ejes fijos, 90° alrededor de x, después 90° alrededor de y y por último 90° alrededor de z. Calcular la matriz de rotación y dónde termina el punto (100, 0, 0) del objeto.

Matriz de transformación homogénea (sección 1.6)

12. El sistema $\{S_1\}$ se gira 90° alrededor de x y después se traslada (20, −30, −80). Escribir ${}^{0}T_{1}$ y calcular las coordenadas en $\{S_0\}$ del punto $\boldsymbol{r}$ = (−30, 90, −10) dado en $\{S_1\}$.

BORRADOR

13. El sistema $\{S_1\}$ se gira −90° alrededor de *y* y después se traslada (50, 0, 100). Escribir ${}^{0}T_1$ y calcular las coordenadas en $\{S_0\}$ del punto (10, 20, 30) dado en $\{S_1\}$.
14. El sistema $\{S_1\}$ se gira 90° alrededor de *z*, después 90° alrededor de *x* (ejes fijos) y por último se traslada (0, 100, 0). Escribir ${}^{0}T_1$ y calcular dónde queda el punto (100, 0, 0) de $\{S_1\}$.

El orden de rotación y traslación (sección 1.6)

15. Con los datos del ejercicio 12, calcular las coordenadas en $\{S_0\}$ del mismo punto si el orden fuera el inverso: primero trasladar y después girar alrededor del eje *x* fijo.
16. Un sistema se gira 90° alrededor de *z* y se traslada 100 mm sobre *z*. Calcular la matriz en los dos órdenes. En este caso dan lo mismo: explicar por qué.

Sucesión de transformaciones (sección 1.6)

17. En el brazo de dos eslabones de la sección 1.6, con $L_1 = 400$, $L_2 = 250$, $q_1 = 90°$ y $q_2 = -90°$, calcular la postura de la punta. Dibujar el brazo para verificar.
18. El mismo brazo con $L_1 = L_2 = 200$ y $q_1 = q_2 = 45°$. Calcular la postura de la punta y comparar la posición con la fórmula trigonométrica.
19. El sistema $\{S_1\}$ se gira 90° alrededor de *z* y se traslada (100, 0, 0) respecto de $\{S_0\}$. Después, respecto de los ejes ya movidos de $\{S_1\}$, se gira 90° alrededor de *u* y se traslada (0, 50, 0), y así se obtiene $\{S_2\}$. Calcular ${}^{0}T_2$ y las coordenadas en $\{S_0\}$ del punto (0, 0, 10) de $\{S_2\}$.

Figuras (secciones 1.5 y 1.6)

20. Un círculo de 20 mm de radio, centrado en el origen y contenido en el plano *xy*, se gira 30° alrededor de *y*. Calcular dónde quedan sus cuatro puntos cardinales (20, 0, 0), (0, 20, 0), (−20, 0, 0) y (0, −20, 0), y dibujar el círculo antes y después.
21. Un rectángulo de 30 × 50 mm, con vértices (0, 0, 0), (30, 0, 0), (30, 50, 0) y (0, 50, 0), se gira −30° alrededor de *x*. Calcular los vértices girados y verificar que los lados conservan su longitud.
22. A un cuadrado se le aplican una rotación de −45° alrededor de *x* y una traslación de −30 mm sobre *y*. Escribir la matriz homogénea en los dos órdenes posibles y dibujar los dos resultados lado a lado con subplot.

Con los sistemas de ejes coordenados físicos (sección 1.6)

23. Con los dos sistemas físicos, girar $\{S_1\}$ 90° alrededor del eje *x* (rojo) del sistema fijo y después 90° alrededor del eje *z* (azul) del sistema fijo. Anotar hacia dónde apunta cada eje del sistema móvil, escribir *R* y comparar con $R(z, 90°)\,R(x, 90°)$.
24. Repetir el ejercicio 23, pero con el segundo giro alrededor del eje *z* (azul) del sistema móvil. Escribir *R*, compararla con la del ejercicio 23 y decidir cuál de los dos productos, $R(z, 90°)\,R(x, 90°)$ o $R(x, 90°)\,R(z, 90°)$, corresponde a cada caso.

Respuestas

Todas verificadas en Octave. Las matrices se escriben por filas, separadas por punto y coma, como en el código.

1. $\mathbf{v}$ = (300, −400, 0); distancia 500 mm; versor (0,6; −0,8; 0).

BORRADOR

2. Ángulo 45°; proyección 100 mm.
3. $\boldsymbol{a} \times \boldsymbol{b}$ = (60000, 0, 0); versor normal (1, 0, 0); área 60000 mm².
4. (230, −60, 50).
5. (400, 150, 0).
6. $R(y, 90°)$ = `[0 0 1; 0 1 0; -1 0 0]`; el punto va a (100, 0, 0).
7. (86,60; 50; 0).
8. (0, 0, 100). La norma de $R(x, 90°)^T - R(x, -90°)$ da 0.
9. $R = R(y, 90°)\ R(z, 90°)$ = `[0 0 1; 1 0 0; 0 1 0]`: *u* apunta según *y*, *v* según *z* y *w* según *x*.
10. $R = R(z, 90°)\ R(y, 90°)$ = `[0 -1 0; 0 0 1; -1 0 0]`: *u* apunta según −*z*, *v* según −*x* y *w* según *y*. Es distinta de la del ejercicio 9.
11. $R = R(z, 90°)\ R(y, 90°)\ R(x, 90°)$ = `[0 0 1; 0 1 0; -1 0 0]`; el punto termina en (0, 0, −100).
12. ${}^{0}T_1$ = `[1 0 0 20; 0 0 -1 -30; 0 1 0 -80; 0 0 0 1]`; el punto queda en (−10, −20, 10).
13. ${}^{0}T_1$ = `[0 0 -1 50; 0 1 0 0; 1 0 0 100; 0 0 0 1]`; el punto queda en (20, 20, 110).
14. ${}^{0}T_1$ = `[0 -1 0 0; 0 0 -1 100; 1 0 0 0; 0 0 0 1]`; el punto queda en (0, 100, 100).
15. (−10, 90, 60), contra (−10, −20, 10) del ejercicio 12.
16. En los dos órdenes, T = `[0 -1 0 0; 1 0 0 0; 0 0 1 100; 0 0 0 1]`. Dan lo mismo porque la traslación es a lo largo del eje de giro: $R(z, 90°)$ deja quieto al vector (0, 0, 100), así que $R\,\boldsymbol{d} = \boldsymbol{d}$.
17. T = `[1 0 0 250; 0 1 0 400; 0 0 1 0; 0 0 0 1]`: la punta está en (250, 400, 0) y con la misma orientación que la base, porque $q_1 + q_2 = 0$.
18. T = `[0 -1 0 141.42; 1 0 0 341.42; 0 0 1 0; 0 0 0 1]`. El bloque de rotación es $R(z, 90°)$; la posición coincide con 200 cos 45° + 200 cos 90° = 141,42 y 200 sen 45° + 200 sen 90° = 341,42.
19. ${}^{0}T_2 = {}^{0}T_1\,{}^{1}T_2$ = `[0 0 1 50; 1 0 0 0; 0 1 0 0; 0 0 0 1]`; el punto queda en (60, 0, 0).
20. (17,32; 0; −10), (0, 20, 0), (−17,32; 0; 10) y (0, −20, 0).
21. (0, 0, 0), (30, 0, 0), (30; 43,30; −25) y (0; 43,30; −25). Los lados siguen midiendo 30 y 50 mm.
22. Rotar y después trasladar: `[1 0 0 0; 0 0.7071 0.7071 -30; 0 -0.7071 0.7071 0; 0 0 0 1]`. Trasladar y después rotar: el mismo bloque de rotación con cuarta columna (0; −21,21; 21,21).
23. R = `[0 0 1; 1 0 0; 0 1 0]`: el eje *x* (rojo) del sistema móvil apunta como el eje *y* (verde) fijo, el eje *y* (verde) como el eje *z* (azul) fijo y el eje *z* (azul) como el eje *x* (rojo) fijo. Coincide con $R(z, 90°)\ R(x, 90°)$.
24. R = `[0 -1 0; 0 0 -1; 1 0 0]`: la roja apunta como la azul fija, la verde al revés que la roja fija y la azul al revés que la verde fija. Es $R(x, 90°)\ R(z, 90°)$: con el segundo giro sobre un eje móvil, la matriz nueva va a la derecha.

BORRADOR